SPARQ+: Structure-Conditioned Evidence Retrieval and Adaptive Reasoning for Offline Open-Domain Table-Text Question Answering

Mengyi Yan, Yang Liu, Weilong Ren*, Yutong Ye, Haoyi Zhou, Jianxin Li, and Zhumin Chen

***Abstract*—Table Question Answering (TQA) increasingly relies on online large language models (LLMs), but such systems often incur privacy risks, high latency, and high deployment cost. Existing offline pipelines either assume a given table or rely on flat similarity-based retrieval, which often misses answer evidence connected through table structure. We present SPARQ+, a cost-efficient framework for offline table–text QA that handles both the closed setting, where the target table is given, and the open setting, where both the relevant table and supporting passages must be discovered before reasoning. SPARQ+ follows a minimal-sufficiency principle: it retrieves tables by fusing lexical, dense, and graph-structural signals, discovers supporting passages through cell-link neighborhoods that convert corpus-wide passage search into budgeted neighborhood lookup, and reasons over the assembled evidence with a passage-aware router, a mutual-information verifier, and rollback/fallback strategies. These components are unified by an information-bottleneck objective that minimizes evidence cost while preserving answer sufficiency. Experiments on seven benchmarks show that SPARQ+ achieves strong effectiveness and efficiency in both settings, improving accuracy by up to 4.0 pp (resp. 8.7 pp) in the closed (resp. open) setting over the strongest baseline, while reducing latency by up to $30.6\times$.**

***Index Terms*—Table Question Answering, Offline Large Language Models, Adaptive Query Routing.**

I. Introduction

Table Question Answering (TQA), a core task in table reasoning, bridges natural language understanding and structured data analysis [1], [2]. It aims to derive concise factual answers, detailed explanations, or fact-checking verdicts from a user's textual query over schema-free tables [3]. The task is inherently challenging, as it requires translating ambiguous natural language into executable reasoning steps that combine logical, numerical, and textual inference over semi-structured tables [4], [5]; the database community has recently studied this paradigm as a natural interface for querying heterogeneous data without predefined schemas [6]–[9]. Despite these difficulties, TQA holds great promise: it empowers non-experts to query and reason over large-scale data using plain language, producing interpretable and verifiable analytical results [2].

Corresponding authors: Weilong Ren.

Yang Liu, Yutong Ye, Haoyi Zhou, and Jianxin Li are with Beihang University, Beijing 100191, China. (E-mail: ly_act@buaa.edu.cn; yutongye@buaa.edu.cn; haoyi@buaa.edu.cn; lijx@buaa.edu.cn).

Mengyi Yan and Zhumin Chen are with Shandong University, Shandong 250100, China. (E-mail: yanmy@sdu.edu.cn; chenzhumin@sdu.edu.cn).

Weilong Ren is with Shenzhen Institute of Computing Sciences, Shenzhen 518100, China. (E-mail: renweilong@sics.ac.cn).

Research on table QA has progressed along two lines: the *closed* and *open* settings. (1) In the *closed* setting, the target table is given and contains the information required to answer the question; the primary challenge is therefore to interpret the table and reason over it effectively. Approaches have evolved from supervised models to LLM-based in-context learning [10], [11] and chain-of-thought prompting [12], [13], and more recently to hybrid frameworks that combine symbolic and semantic reasoning [14]–[16]. (2) In the *open* setting, the relevant table is not known a priori and must be identified, often together with supporting passages, from a large corpus. Research has evolved from dense table retrieval and early-fusion block construction to multi-hop, chain-of-skills retrieval and graph-structured evidence discovery [17], [18]. Across both settings, recent state-of-the-art (SOTA) systems increasingly adopt LLMs as their primary reasoning engine.

These two lines have largely advanced in isolation: open-setting methods improve evidence retrieval but typically delegate downstream reasoning to a generic reader, whereas closed-setting methods support fine-grained operator-level reasoning but assume that the required evidence is already available. Real-world queries, however, often require both: discovering the right evidence from a heterogeneous corpus *and* reasoning precisely over the discovered evidence. Neither line alone fully addresses this need.

Moreover, most table QA systems are built around *online LLM services* (*i.e.,* cloud-hosted models), with comparatively less attention to *offline LLMs* deployed locally. This bias is increasingly problematic as practitioners seek lower cost and latency, stronger privacy, and easier on-premise deployment [19], [20]. A naive remedy is to directly replace online models with smaller offline ones, but such substitutions often degrade answer quality.

Challenges. This unified view highlights two challenges that existing approaches, typically designed for a single pipeline stage in isolation, fail to address, especially when coupled with offline LLMs. *(C1) Similarity alone is insufficient for evidence discovery.* In open table QA, textual or semantic similarity is often insufficient for locating the evidence needed to answer a query. Many retrievers linearize tables into token sequences, obscuring row–column topology and weakening relational signals [8], [21], [22]. Sparse retrievers such as BM25 [23] further treat numeric values, headers, and cell contents as ordinary tokens. More importantly, answer-bearing

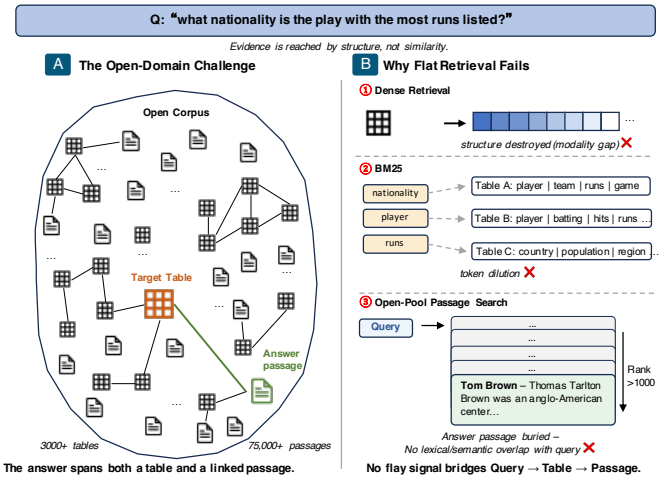


Fig. 1: Retrieval challenge (C1): the answer passage is reached through the table's cell-link topology, not query–passage similarity.

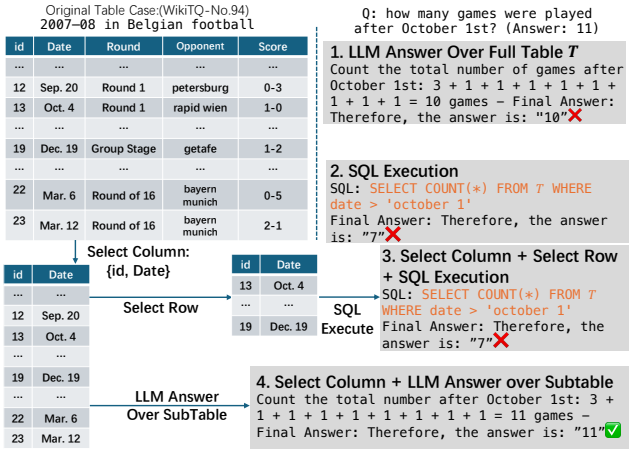


Fig. 2: Reasoning challenge (C2): different operators yield different answers over the same table; only the minimally sufficient set is correct.

passages may be reachable only through cell-level hyperlinks, with little or no direct overlap with the query [24], [25]. In such cases, query–passage similarity is unreliable, and the table's link structure becomes essential for evidence discovery. *(C2) More evidence is not necessarily better evidence.* Even after relevant tables or passages are retrieved, feeding an entire table to an offline LLM can hurt accuracy, as redundant or irrelevant context may distract the model. Moreover, different operators over the same table can yield different answers; only those required by the query should be applied. Overall, effective table QA requires evidence that is sufficient, but no larger than necessary, for both discovery and reasoning.

We illustrate both challenges with the following example.

Example 1: Consider the query *"What nationality is the player with the most runs listed?"* over a corpus containing 3,000+ tables and 75,000+ passages. *Retrieval (C1, Fig. 1).* The answer, "Anglo-American," appears in a passage that is connected to the query only through the hyperlink in the table cell "Tom Brown." Under open-pool retrieval, this passage is ranked below 1,000 among 75,000 candidates. In contrast, restricting retrieval to the cell-link neighborhood of the retrieved table reduces the search space to ∼15 passages, allowing the answer-bearing passage to surface naturally. *Reasoning (C2, Fig. 2).* Given the retrieved table, feeding the full table to the LLM misidentifies the player, while a single SQL query cannot bridge from the table to the linked passage. In contrast, selecting only the "Player" and "Runs" columns, identifying the player with the maximum runs, and then reading the linked passage yields the correct answer. This illustrates a minimally sufficient combination of evidence and operations.

Our approach. SPARQ+ addresses these challenges with a three-stage pipeline that progressively narrows the search space and reasons over only minimally sufficient evidence. *(1) Multi-signal table retrieval* first retrieves a small set of candidate tables by combining structural, lexical, and semantic signals, and then reranks these candidates with a cross-encoder to identify the most suitable table. *(2) Structure-conditioned passage discovery* subsequently searches for supporting passages within the cell-link neighborhood of the retrieved table, rather than over the entire corpus, thereby reducing the candidate pool by orders of magnitude. *(3) Passage-aware reasoning* finally selects and executes a minimally sufficient set of operators over the assembled evidence, while a passage-aware verifier enforces the evidence budget and triggers rollback when necessary.

Contributions. This paper presents SPARQ+ (Sufficient and Precise Adaptive Routing for Open-Domain Table Question Answering+), a cost-efficient framework for offline open-domain table–text reasoning on consumer-grade hardware. We summarize the main contributions as follows.

(1) Preliminary analysis (Section II). Building on the closed-setting analysis of SPARQ, we conduct controlled experiments that isolate reasoning and retrieval failures in offline TQA. The analysis shows that more context and longer reasoning chains do not necessarily improve offline LLMs, and that answer-bearing passages are often discoverable through table topology rather than direct query–passage similarity. These findings motivate a minimal-sufficiency design that controls both evidence discovery and reasoning cost.

(2) Closed-setting reasoning core (Section III). We study offline table reasoning when the target table is given and present the closed-setting SPARQ core. SPARQ formulates operator selection as a cost-aware minimal-sufficiency problem, uses a modular pool of semantic and symbolic operators, and couples an adaptive router with a mutual-information verifier. The verifier checks whether the selected subtable and execution context retain sufficient evidence, and triggers rollback or fallback when the reasoning context is under-specified.

(3) Open-domain TQA (Section IV). We extend SPARQ to the open setting, where both the relevant table and supporting passages must be discovered from a corpus. SPARQ+ first performs multi-signal table retrieval by fusing lexical, dense, and graph-structural signals, and then conducts structure-conditioned passage discovery within the retrieved table's cell-link neighborhood, turning corpus-wide passage search into budgeted neighborhood lookup. It further extends the router and verifier to support passage-aware, rollback-aware

reasoning over progressively assembled table–passage evidence, yielding a unified information-bottleneck objective that minimizes evidence cost while preserving answer sufficiency.

(4) Comprehensive evaluation (Section V). We evaluate SPARQ+ on seven benchmarks covering both closed-setting and open-domain TQA. In the closed setting, SPARQ+ preserves the effectiveness and latency benefits of SPARQ. In the open setting, SPARQ+ achieves the best EM/F1 among the compared offline/open-domain baselines on HybridQA and OTT-QA, improving EM by up to 8.7 pp over the strongest open-domain baseline and by 17.2 pp over COS on OTT-QA, while reducing latency by up to 30.6× compared with iterative pipelines. Ablations and diagnostics further show that SPARQ+ improves by assembling more sufficient evidence, and that long-form reasoning amplifies evidence quality rather than compensating for missing or incorrect evidence.

Extension over the conference version. This journal article substantially extends SPARQ [26], our offline closed-setting reasoner with adaptive routing and sufficiency verification. It introduces SPARQ+ for both closed- and open-domain table–text QA, reducing to SPARQ when the corpus contains one table and no passages. Specifically, we (1) analyze the interaction between evidence discovery and reasoning and unify them under an information-bottleneck objective (Sections II-B and IV-A); (2) develop multi-signal table retrieval with a query-conditioned heterogeneous GNN and structure-conditioned passage discovery with theoretical guarantees (Sections IV-B and IV-C); (3) extend the router and verifier with passage-aware, budgeted, and rollback-aware reasoning (Section IV-D); and (4) evaluate SPARQ+ on seven benchmarks, improving open-domain EM by up to 8.7 pp and reducing latency by up to 30.6×, while preserving the closed-setting benefits of SPARQ (Sections V-C and V-E). Proofs and additional results can be found in the full version [27].

II. Motivation and Preliminary Analysis

This section presents a preliminary analysis of offline table-text QA to identify the key bottlenecks in both closed and open settings that guide our method design. We focus on offline deployment, where a local LLM answers queries by applying lightweight table operations to construct compact contexts. Through controlled experiments, we first examine reasoning failures when the target table is given (Section II-A), and then analyze retrieval failures over a large open-domain corpus (Section II-B). The former motivates our closed-setting solution (Section III), while the latter motivates our open-setting extension (Section IV).

A. Reasoning Failures

Setup. We use WikiTQ [5] and select 476 hard samples following H-STAR [14], each with more than 4,000 table tokens and averaging 78.5 rows and 7.73 columns. We evaluate eleven methods under a unified offline setting: seven single-operator methods (Base, Select_Column, Select_Row, Execute_SQL, Execute_Py, RAG, RAG + Rewrite), two SOTA hybrid methods (H-STAR [14], ReAcTable [7]), and two prototype routing methods (Route, Route + Check). All methods use Qwen3-4B-Instruct [28] as the offline LLM, three-shot CoT prompting, and two RTX 4090 GPUs.

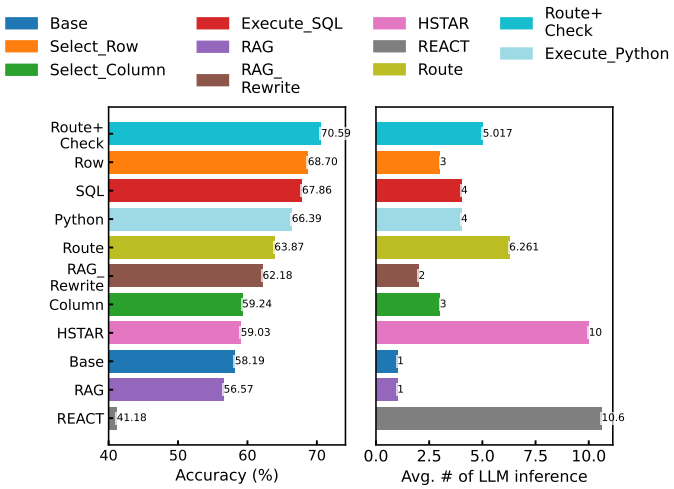


Fig. 3: Reasoning analysis on WikiTQ hard samples (476 queries). Each point represents one method; the x-axis shows accuracy, the y-axis shows average LLM calls.

Observations. As shown in Fig. 3, we find the following.

(1) Offline LLMs struggle with long-form tables. Base, which feeds the full table to the LLM, achieves only 0.582 accuracy. Compared with Base, Select_Row improves accuracy by 18.06% by reducing the input size, while Execute_SQL improves accuracy by 16.62% through symbolic computation.

(2) Symbolic execution is powerful but brittle. Although SQL and code-based operators improve accuracy by up to 18.06% (as in (1)), their internal failure rates reach 30.30%, due to syntax errors and dirty data, which limits further improvement.

(3) Overthinking wastes resources without improving accuracy. Compared with Select_Row, H-STAR and ReAcTable require 3.3× and 3.5× more LLM calls, respectively, yet suffer accuracy drops of 14.08% and 40.06%. This suggests that excessive reasoning steps can not only waste computation, but also discard or obscure necessary information.

(4) TQA benefits from minimal yet sufficient information. Route + Check improves accuracy over Base by 21.31%, while reducing inference time by 19.86% with only 1.26 operators on average. However, its reliance on manual verification makes it impractical at scale.

B. Retrieval Failures

In Section II-A, we assume that the target table is given. We now relax this assumption and examine what happens when the system must discover supporting evidence from a large corpus.

Setup. We use the development split of OTT-QA [29] (1,690 queries, 8,891 tables, and 240,042 passages), an open-domain benchmark over tables and text, where answering a query typically requires retrieving a table and hopping to its linked passages; the corpus provides a bipartite graph connecting table cells and passages via 277K cell-to-passage links. For

TABLE I: Passage quality ladder on OTT-QA

Configuration	Passage source	K_{pas}	EM
No-passage	$\emptyset$	0	0.132
Open-pool	BM25 over 240K corpus	15	0.239
Cell-link	BM25 within $\mathcal{N}(T^*)$	15	0.639
Gold (oracle)	annotated gold passages	–	0.742
CORE-B [30]	flat retrieval over 240K	15	0.169
ColBERTv2-B [31]	flat retrieval over 240K	15	0.240

TABLE II: Table Recall@K on OTT-QA.(Reranker over RRF K@20)

Method	K@**1**	K@**5**	K@**10**	K@**20**
BM25	0.666	0.825	0.874	0.923
Dense	0.742	0.895	0.934	0.959
GNN	0.736	0.909	0.947	0.965
RRF (3-way)	0.808	0.936	0.961	0.977
+ Reranker	0.859	0.941	0.963	0.977

passage quality analysis, we focus on the *passage-recoverable* subset (1,199 queries whose gold answers appear in at least one linked passage), fix the gold table, and vary only the passage source, thereby isolating passage quality from table retrieval noise. We compare four passage configurations (Table I): (a) No-passage provides the gold table but supplies no passages; (b) Open-pool retrieves BM25 passages from the full 240K-passage corpus; (c) Cell-link restricts candidates to passages linked to the gold table (via cell-to-passage links) before ranking; and (d) Gold (oracle) uses gold passage IDs. Here K_{pas} denotes the number of passages given to the reader, and $\mathcal{N}(T^*)$ the passages cell-linked to the gold table T^*. For table retrieval, we additionally report Recall@K over retrieved tables under both single-signal and fused settings. All results use Qwen3.6-35B [28] as the reader, with end-to-end QA performance measured by Exact Match Accuracy(EM).

Observations. As shown in Tables I–III, we find the following.

(5) Flat retrieval hits a structural ceiling. BM25 and dense retrieval achieve comparable table-retrieval performance (R@1: 0.666 vs. 0.742), while the GNN alone reaches 0.736 using only structural cues (Table II). Fusing the three signals via RRF raises R@1 to 0.808, and reranking further to 0.859, showing that structural cues complement flat textual signals.

(6) Passage reachability is structural, not purely semantic. Full-corpus query–passage retrieval yields only modest gains (EM: 0.239 vs. 0.132 without passages), consistent with prior open-domain baselines (Table I), as the answer-related passage is often reachable only via a cell link rather than textual overlap with the query. Restricting candidates to cell-linked passages boosts EM to 0.639, with diminishing returns beyond K_{pas}=15, suggesting that a small passage budget suffices once structural links are exploited (Tables I and III).

(7) Tables alone rarely suffice for open-domain queries. Even with the oracle table, using no passages achieves only 0.132 EM on passage-recoverable queries (Table I). Providing gold passages raises EM to 0.742, confirming that linked passages are necessary evidence rather than optional context.

(8) Retrieval and reasoning failures compound. With the same reader, changing only the evidence pipeline shifts EM from 0.132 to 0.742, a 5.6× range (Table I). This shows that retrieval and reasoning are tightly coupled: even a strong reasoner cannot recover missing evidence, while high-quality passages are ineffective if the wrong table is retrieved.

TABLE III: Cell-link passage budget sweep.

K	5	15	40	80
EM	0.515	0.639	0.687	0.700

C. Design Implications

Together, these observations motivate the key design features of SPARQ+.

1) *Adaptive operator routing with verification* (Observations 1–4). In closed-setting table QA, no single operator is consistently effective; SPARQ+ therefore uses a cost-aware router to select the reasoning path (Section III-C) and a verifier to ensure information sufficiency (Section III-D).
2) *Multi-signal table retrieval with structural fusion* (Observation 5). In open-domain table retrieval, flat textual signals saturate; SPARQ+ therefore fuses lexical, dense, and structural signals, using a GNN to inject topological evidence (Section IV-B).
3) *Structure-conditioned passage discovery* (Observation 6). In open-domain passage retrieval, answer-bearing evidence is often reachable via table hyperlinks rather than query–passage similarity alone; SPARQ+ thus prioritizes retrieval from the cell-link neighborhood, with a union-pool fallback and an adaptive passage budget (Section IV-C).
4) *Passage-aware reasoning* (Observation 7). In open-domain table QA, tables alone rarely suffice; SPARQ+ therefore extends its offline reasoning pipeline to incorporate linked passage evidence (Section IV-D).
5) *Unified retrieval–reasoning optimization* (Observation 8). In end-to-end open-domain table QA, retrieval and reasoning failures compound; SPARQ+ therefore jointly optimizes evidence discovery and reasoning under a unified minimal-sufficiency objective (Section IV-A).

III. Reasoning over a Given Table

In this section, we study the *closed* offline setting, where the target table T is given and queries must be answered without overwhelming the LLM. We present SPARQ, an offline table-QA framework built on *minimal sufficiency*: we formalize the problem and the notion of minimal sufficiency (Section III-A), define an operator pool (Section III-B), route each query to a minimally sufficient operator set to construct a compact context (Section III-C), and verify answer sufficiency (Section III-D). We summarize the complete closed-setting system in Section III-E, and extend this core to the *open* setting in Section IV, where the relevant table and supporting passages must be discovered from a corpus.

A. Problem and Minimal Sufficiency

Problem. Given a query–table pair (q, T) and an offline LLM, SPARQ selects an operator set $\mathcal{S} \subseteq \mathcal{O}$ from a typed pool $\mathcal{O}$ (Section III-B) and applies it to T. This produces a derived

subtable $T^{\mathcal{S}} \subseteq T$ and an LLM-generated context $\mathsf{Context}(\mathcal{S})$, from which SPARQ outputs the answer a to q.

Minimal sufficiency. Given a query–table pair (q, T) and a pretrained LLM, a subtable $T^* \subset T$ is *minimally sufficient* if it (a) contains the information necessary to answer q correctly (sufficiency), and (b) minimizes the token length required by the LLM (minimality). However, identifying such a subtable is extremely challenging, as it requires enumerating all possible row–column combinations of T and evaluating each candidate with the LLM, leading to an exponential complexity of $O(2^{m+n} \cdot \mathrm{COST}(\mathsf{LLM}))$, where m and n denote the number of rows and columns in T, respectively, and $\mathrm{COST}(\mathsf{LLM})$ represents the cost of LLM inference on a single subtable. Ambiguous queries or overly long or incomplete tables usually incur a high $\mathrm{COST}(\mathsf{LLM})$ and often lead to error propagation.

This leads to the closed-setting objective:

$$\min_{\mathcal{S} \subseteq \mathcal{O}} \ \mathrm{COST}(\mathcal{S}) \quad \text{s.t.} \quad Q(q, T^{\mathcal{S}}, \mathsf{Context}(\mathcal{S})) \geq \tau, \qquad (1)$$

where $\mathrm{COST}(\mathcal{S})$ denotes the inference cost of executing $\mathcal{S}$, Q is the mutual-information verifier in Section III-D, and τ is a sufficiency threshold.

B. An Extended Pool of Semantic and Symbolic Operators

Table operators. SPARQ uses a modular operator pool $\mathcal{O}$ consisting of five operators for compositional table reasoning. By combining semantic and symbolic processing, it avoids the brittleness of one-shot generation [13], which often causes syntax or execution errors on offline LLMs. (a) The preprocessing operator o^{prep} normalizes raw tables and produces lightweight statistical summaries, yielding compact and noise-reduced inputs. (b) The column extraction operator o^{col} identifies query-relevant columns using a dual-path strategy that combines LLM-based enumeration with SQL-based selection, thereby reducing hallucinations. (c) The row extraction operator o^{row} applies the same dual-path strategy to identify query-relevant rows. (d) The retrieval operator o^{Ret} provides RAG-style evidence access within a table: it rewrites q into multi-granular variants (general, balanced, and specific) [32], and combines dense SBERT embeddings [33] with sparse BM25 features [23] using Weighted Reciprocal Rank Fusion [34] to select the top-M rows and top-N columns. (e) The SQL execution operator o^{SQL} performs symbolic computation and improves robustness by pruning infeasible execution paths.

Overall, this modular design bridges semantic retrieval and symbolic reasoning, enabling efficient evidence access and computation without full-table prompting. Operator prompts are provided in Appendix D.

Remark. Unlike conventional TQA pipelines that follow fixed symbolic or neural reasoning paths, SPARQ uses a modular operator design for flexible composition and self-correction. In particular, the retrieval operator o^{Ret} bridges semantic retrieval and symbolic computation (Section II), enabling offline LLMs to access contextual evidence efficiently, a capability typically associated with online RAG systems.

C. Adaptive Query Router E

Challenge. Given a query–table pair (q, T) and an offline LLM, multiple operator sets may produce the same correct answer. For example, a query may be answered by selecting relevant rows with o^{row}, retrieving semantically related cells with o^{Ret}, or composing row selection with symbolic execution, e.g., $\{o^{\mathsf{row}}, o^{\mathsf{SQL}}\}$. However, these operators incur different costs: o^{SQL} requires three LLM generations; o^{row} and o^{col} each require two; and o^{Ret} rewrites the query into three variants and matches each variant against all m rows and n columns, resulting in approximately $3(m+n)$ similarity computations plus one LLM generation for rewriting. Thus, operator routing must balance information sufficiency against execution cost.

Mechanism. We define $\mathrm{COST}(q, T, \mathcal{S})$ as the inference cost of answering q over T using operator set $\mathcal{S}$, which depends on both the logical complexity of q and the structural scale of T. Although multiple operator sets may yield the correct answer, the optimal one is the lowest-cost set that remains sufficient for correctness. This optimum is unknown a priori, as it depends on the interaction between query logic and table content. Thus, SPARQ adopts an approximate-and-verify design: the router E predicts an operator set $\mathcal{S}$ to approximate the cost-minimal sufficient set, while the verifier Q (Section III-D) checks sufficiency to avoid under-approximation. For each operator $o_i \in \mathcal{S}$, we define its fitness probability under the router's predicted distribution over set choices:

$$o_i.p = \sum_{\mathcal{S}_k :\, o_i \in \mathcal{S}_k} P(\mathcal{S}_k \mid q, T), \qquad (2)$$

which aggregates confidence over all candidate reasoning paths containing o_i. Once $\mathcal{S}$ is selected, Q evaluates its sufficiency and triggers rollback when necessary, thereby instantiating the constrained objective in Eq. (1).

Reward. To train E, SPARQ samples a small set of hard training tables T^{train} with long contexts (> 4K tokens) and ground-truth labels l, and enumerates a cost-ordered operator-set space $\boldsymbol{\mathcal{S}} = \{\mathcal{S}_0, \mathcal{S}_1, \ldots, \mathcal{S}_k\}$, where $\mathcal{S}_0$ denotes the base case (no operator applied) and $\mathcal{S}_k$ the most expensive configuration. Querying the offline LLM with triplets $(q, \mathsf{Context}(\mathcal{S}), T^{\mathcal{S}})$ yields predictions p, which are scored as

$$\mathcal{R}(q, \mathcal{S}, T^{\mathcal{S}}) = \mathsf{acc}(p, l) \times \rho_{\mathrm{cost}}(\mathcal{S}) \times \rho_{\mathrm{len}}(\mathcal{S}), \qquad (3)$$

where $\mathsf{acc}(p, l)$ measures answer accuracy, $\rho_{\mathrm{cost}}(\mathcal{S}) = \exp(-\lambda_c \cdot \mathrm{COST}(\mathcal{S}))$ penalizes expensive operator sets, and $\rho_{\mathrm{len}}(\mathcal{S})$ penalizes long contexts.

Router design. SPARQ supports two routers. The *training-free* router uses in-context prompting to let a pre-trained LLM select the most suitable operator set from $\mathcal{O}$ for a given (q, T), providing flexibility without training overhead. The *training-based* router offers a lightweight alternative when the training-free router is insufficiently accurate. Using the reward in Eq. (3) as soft labels, we train E as a multi-class classifier to predict the operator-set distribution $P(\mathcal{S} \mid q, T)$. We cast this process as knowledge distillation and further add an InfoNCE [35] loss to contrast high- and low-reward

operator sets, steering E toward cost-efficient reasoning paths.

Remark. An operator set that is too small may leave the LLM with an overly long context, while an overly aggressive set may filter out critical evidence; both can degrade accuracy. We thus introduce the verifier Q next to mitigate this trade-off.

D. Mutual-Information Verifier Q

Motivation. As mentioned above, an overly large operator set $\mathcal{S}$ may filter out critical evidence and hurt accuracy. To mitigate this trade-off, SPARQ introduces a mutual-information verifier Q that checks, in real time, whether the subtable $T^{\mathcal{S}}$ (and intermediate result $\mathsf{Context}(\mathcal{S})$) produced by executing $\mathcal{S}$ on T retains sufficient information to answer q.

Sufficiency score. Given (q, T), a subtable $T^{\mathcal{S}}$, and its execution result $\mathsf{Context}(\mathcal{S})$, Q computes a sufficiency score by aggregating semantic alignment across modality pairs:

$$Q(q,T^{\mathcal{S}},\mathsf{Context}(\mathcal{S})) = \sigma\big(\alpha f_\theta(q,T^{\mathcal{S}}) + \beta f_\theta(q,\mathsf{Context}(\mathcal{S})) + \gamma f_\theta(T^{\mathcal{S}},\mathsf{Context}(\mathcal{S}))\big). \quad (4)$$

where σ is the sigmoid function, f_θ is a Transformer-based cross-encoder returning a scalar in $[-1, 1]$, and $\alpha = \beta = 1, \gamma = 0.2$ by default. The three terms capture query–subtable, query–execution, and subtable–execution alignment; higher Q indicates greater evidence sufficiency.

Rollback strategy. If $Q < \tau$, SPARQ performs rollback by expanding $T^{\mathcal{S}}$ to a larger subtable $T^{\mathcal{S}'}$, where $\mathcal{S}' = \mathcal{S} \setminus \{o_{\min}\}$ and $o_{\min}$ is the extraction operator with the lowest fitness probability $o_i.p$. The verifier then re-evaluates $T^{\mathcal{S}'}$ iteratively until it finds a sufficient context or reaches the base case $\mathcal{S}_0$ (*i.e.,* no operator applied). Rollback mitigates multi-step error accumulation without LLM regeneration and applies only to extraction operators (*i.e.,* o^{row}, o^{col}, and o^{Ret}).

Fallback strategy. If the base case still lacks sufficient evidence, SPARQ falls back to the full table T and prompts the LLM to generate an SQL query for retrieving additional support, which is added to the final inference context. This safeguard is used only as a last resort, since SQL parsing failures in such cases are typically harder to recover from.

Example 2: Suppose that the router over-selects $\{o^{\mathsf{col}}, o^{\mathsf{row}}, o^{\mathsf{SQL}}\}$ and o^{row} erroneously filters the target row, yielding $T^{\mathcal{S}} = \emptyset$. The verifier detects $Q < \tau$ and rolls back to $\{o^{\mathsf{col}}, o^{\mathsf{SQL}}\}$, removing o^{row} as the operator with the lowest fitness probability, which restores the missing evidence. For a different query where neither o^{row} nor the base case suffices, SPARQ triggers the fallback, prompting the LLM to generate an SQL query over T to retrieve supplementary evidence before the final answer.

Training. Q is trained with cross-modal contrastive learning over three pairs: $(q, T^{\mathcal{S}})$, $(q, \mathsf{Context}(\mathcal{S}))$, and $(T^{\mathcal{S}}, \mathsf{Context}(\mathcal{S}))$. For each pair (X, Y), we optimize the loss:

$$\mathcal{L}_{XY} = -\log \frac{\exp(s^{XY+})}{\exp(s^{XY+}) + \sum_{j=1}^{k} \exp(s_j^{XY-})}, \quad (5)$$

where s^{XY+} is the positive-pair score and $\{s_j^{XY-}\}_{j=1}^{k}$ are negative-pair scores. Positive pairs are drawn from high-reward operator sets, while negatives are constructed from mismatched pairs and corrupted subtables. The final objective is a weighted combination of the three losses using α, β, and γ.

E. The SPARQ System

Workflow. SPARQ separates model preparation from query execution. (1) In the preparation phase, it samples hard validation tables, collects LLM predictions over candidate operator sets as self-distillation supervision, and trains the lightweight router E and verifier Q. (2) At query time, given (q, T), SPARQ preprocesses T with o^{prep}, uses E to select an operator set $\mathcal{S}$, executes $\mathcal{S}$ with the LLM to produce $\mathsf{Context}(\mathcal{S})$, and applies Q to assess sufficiency and roll back when needed, yielding the final subtable $T^{\mathcal{S}}$. It then queries the LLM with $(q, T^{\mathcal{S}}, \mathsf{Context}(\mathcal{S}))$ to produce the answer a. By coupling E with Q, SPARQ approximates a minimally sufficient subtable, enabling small offline models to reason effectively over structured data without relying on online LLMs.

Execution scheduling. To run efficiently on commodity hardware, SPARQ overlaps CPU-bound table operations (*e.g.,*, preprocessing and SQL execution) with GPU-bound LLM inference using a decoupled producer–consumer architecture, and batches queries with similar estimated costs to improve KV-cache reuse. Please see Appendix G and [27] for details.

IV. Evidence Discovering and Reasoning

In this section, we extend the closed setting of Section III to an *open* setting, where the system must discover an evidence set $(T, \mathcal{P}, \mathcal{S})$ from a large corpus before performing table–text reasoning. Specifically, Section IV-A formalizes the corpus, task, and a unified information-bottleneck objective; Section IV-B retrieves the table by fusing lexical, semantic, and structural signals; Section IV-C discovers supporting passages from the retrieved table's cell-link neighborhood under a budget; and Section IV-D extends the SPARQ router and verifier for passage-aware operator routing and sufficiency verification.

A. Problem and Unified Objective

New challenge under open setting. Moving from the closed to the open setting introduces a new mismatch: the table that anchors reasoning and the passage containing the answer may share little surface similarity. Instead, they are often connected through a *cell-level hyperlink* (Observations 6–7 in Section II), implying that evidence discovery should follow structural links rather than rely solely on query–text similarity.

From hyperlinks to a graph. To capture this structure, we represent the corpus as a bipartite graph. A relational web table T consists of (a) a title T.title, (b) headers $T.H = \{h_1, \ldots, h_m\}$, and a (c) cell matrix $T.C = \{c_{i,j}\}_{i\in[n], j\in[m]}$. If a cell $c_{i,j}$ (or the title) hyperlinks to a Wikipedia page, the page's lead paragraph is treated as a passage p. Given a table corpus $\mathcal{T}_{\text{pool}} = \{T_1, \ldots, T_N\}$ and passage corpus $\mathcal{P}_{\text{pool}} = \{p_1, \ldots, p_M\}$, these links induce a bipartite graph $\mathcal{G} = (\mathcal{V}_T \cup \mathcal{V}_P, \mathcal{E}_G)$:

$$(T, p) \in \mathcal{E}_G \iff \exists\, c \in T.C \cup \{T.\text{title}\} \text{ hyperlinking to } p. \quad (6)$$

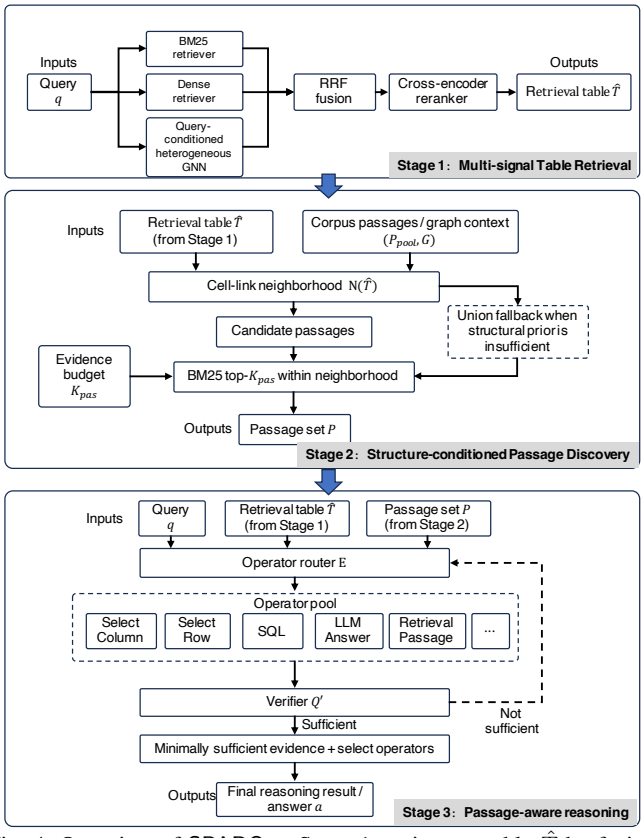


Fig. 4: Overview of SPARQ+. Stage 1 retrieves a table $\hat{T}$ by fusing lexical, semantic, and structural retrieval signals. Stage 2 discovers passages $\mathcal{P}$ from the cell-link neighborhood of $\hat{T}$ under evidence budget K_{pas}. Stage 3 performs passage-aware routing and verification over $(q, \hat{T}, \mathcal{P})$ to produce the answer.

The *cell-link neighborhood* $\mathcal{N}(T) = \{p \in \mathcal{V}_P : (T, p) \in \mathcal{E}_G\}$ serves as the search space for structure-conditioned passage discovery (see Section IV-C).

Problem. We extend SPARQ from the closed to the open setting, denoted as SPARQ+. Given a query q and a corpus $(\mathcal{T}_{\text{pool}}, \mathcal{P}_{\text{pool}}, \mathcal{G})$, SPARQ+ selects an evidence tuple $(T, \mathcal{P}, \mathcal{S})$, including (a) a table $T \in \mathcal{T}_{\text{pool}}$, (b) a passage subset $\mathcal{P} \subseteq \mathcal{P}_{\text{pool}}$, and (c) an operator set $\mathcal{S} \subseteq \mathcal{O}$, to produce an answer a. The closed setting is recovered when $|\mathcal{T}_{\text{pool}}| = 1$ and $\mathcal{P}_{\text{pool}} = \emptyset$.

Unified objective. Both the closed and open settings follow the principle of *minimal sufficiency*, *i.e.,* using the smallest evidence that still suffices to answer q. We formalize this principle using the information bottleneck (IB) [36], [37]:

$$\min_{Z} I(Z; X) - \lambda I(Z; Y), \tag{7}$$

where $I(\cdot;\cdot)$ denotes mutual information, $X = (\mathcal{T}_{\text{pool}}, \mathcal{P}_{\text{pool}})$ is the corpus, $Y = a$ is the answer, and $Z = (T, \mathcal{P}, \mathcal{S})$ is the evidence selected for query q. Instantiating IB in constrained form gives the SPARQ+ objective:

$$\min_{T,\mathcal{P},\mathcal{S}} \text{COST}(T, \mathcal{P}, \mathcal{S}) \quad \text{s.t.} Q'(q, T^{\mathcal{S}}, \mathcal{P}, \mathsf{Context}(\mathcal{S})) \geq \tau, \tag{8}$$

where COST measures evidence compression, Q' is the extended sufficiency verifier (Section IV-D), and τ is a threshold. We decompose cost as $\text{COST}_{\text{ret}}(T, \mathcal{P}) + \text{COST}_{\text{rea}}(\mathcal{S}, T, \mathcal{P})$. Since retrieval incurs largely fixed overhead, the main controllable trade-off lies in COST_{rea}, which grows with the subtable size $|T^{\mathcal{S}}|$ and the number of passages $|\mathcal{P}|$.

Remark. Eq. (8) governs both retrieval and reasoning. Its compression term favors smaller evidence across table retrieval, passage discovery, and operator selection, while the shared verifier Q' enforces sufficiency throughout the pipeline. When the corpus contains a single table and no passages, SPARQ+ reduces to the closed setting of SPARQ: the discovery variables vanish, Q' reduces to Q, and Eq. (8) recovers Eq. (1).

Example 3: For the query *"What nationality is the player with the most runs listed?"* (Example 1), the gold evidence consists of table T^* ("1881 St. Louis Brown Stockings season") and passage p^* (the "Tom Brown" biography), with $p^* \in \mathcal{N}(T^*)$. The answer "Anglo-American" appears only in p^*, which is reachable through the hyperlink on the cell "Tom Brown". Thus, the minimally sufficient evidence is $(T^*, p^*, \mathcal{S}^*)$, where $\mathcal{S}^* = \{o^{\mathsf{col}}\}$ selects only the "Player" and "Runs" columns.

Directly optimizing Eq. (8) entails an expensive joint search over $(T, \mathcal{P}, \mathcal{S})$. We therefore adopt a stagewise approximation: we first retrieve a table $\hat{T}$ (Stage 1) and then discover passages within its cell-link neighborhood $\mathcal{N}(\hat{T})$ (Stage 2), returning a set $\mathcal{P}$ with $|\mathcal{P}| \leq K_{\mathsf{pas}}$, where K_{pas} is the passage budget.

B. Multi-Signal Table Retrieval

Inputs and outputs. Given a query q, a table corpus $\mathcal{T}_{\text{pool}}$, and a graph $\mathcal{G}$, this stage returns the most relevant table $\hat{T}$ together with its cell-link neighborhood $\mathcal{N}(\hat{T})$.

Three retrieval signals. As discussed in Observation 5 (Section II-B), no single retrieval signal suffices: sparse matching ignores relational structure, whereas dense encoders linearize tables and suffer from a modality gap with passage text. We therefore combine three complementary signals: lexical, semantic, and structural, where the structural channel injects cell-link neighborhood information via a query-conditioned GNN. Specifically, each table T is scored along three channels.

Sparse signal. This signal applies BM25 to a serialized table:

$$\begin{aligned} s_{\mathsf{bm25}}(q, T) &= \text{BM25}(q, \text{serialize}(T)), \\ \text{serialize}(T) &= T.\text{title} \oplus T.H \oplus T.C \oplus T.\text{slug}, \end{aligned} \tag{9}$$

where $\oplus$ concatenates the title, headers, cells, and URL slugs.

Dense signal. The dense signal computes the inner product between query and table embeddings:

$$s_{\mathsf{dense}}(q, T) = \text{Embed}_q(q)^\top \text{Embed}_d(\text{serialize}(T)). \tag{10}$$

Structural signal. The structural signal $s_{\mathsf{gnn}}(q, T)$ is produced by a query-conditioned heterogeneous GNN over the bipartite graph $\mathcal{G}$ (Eq. (6)). Table and passage nodes are initialized as

$$\mathbf{h}_T^{(0)} = W_T \text{Embed}_d(\text{serialize}(T)), \mathbf{h}_P^{(0)} = W_P \text{Embed}_d(p), \tag{11}$$

where $W_T, W_P \in \mathbb{R}^{h \times d}$.

To inject neighborhood evidence into the table representation, we run L layers of Heterogeneous Graph Transformer (HGT) convolutions [38] on $\mathcal{G}$, propagating messages over $\mathcal{E}_G$:

$$\mathbf{h}^{(l+1)} = \text{HGTConv}(\mathbf{h}^{(l)}, \mathcal{E}_G), \tag{12}$$

so each table node aggregates information from its cell-linked

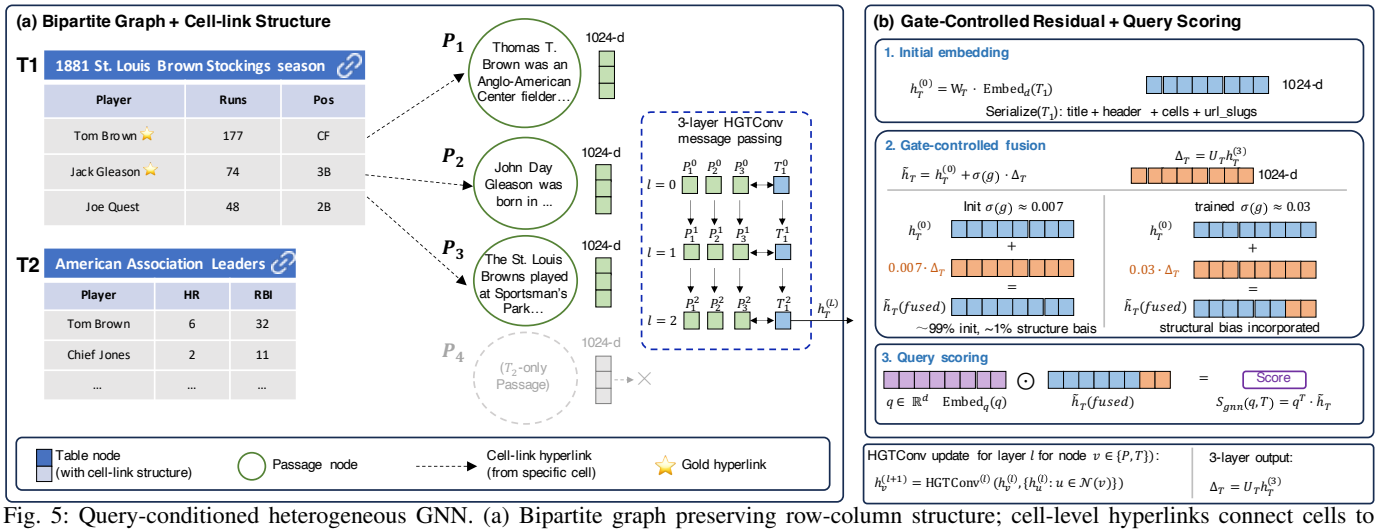


Fig. 5: Query-conditioned heterogeneous GNN. (a) Bipartite graph preserving row-column structure; cell-level hyperlinks connect cells to passage nodes. (b) Gate-controlled residual fusion and query scoring.

passage neighbors to compute $s_{\text{gnn}}(q, T)$.

We refine the resulting representation with a gated residual:

$$\tilde{\mathbf{h}}_T = \mathbf{h}_T^{(0)} + \sigma(g) U_T \mathbf{h}_T^{(L)}, \tag{13}$$

where the gate is initialized near zero, so that structural information is incorporated only when beneficial.

We project the query embedding as $\mathbf{q} = W_q \,\text{Embed}_q(q)$ and compute the structural score as

$$s_{\text{gnn}}(q, T) = \mathbf{q}^\top \tilde{\mathbf{h}}_T. \tag{14}$$

Training. The GNN is trained with an InfoNCE loss:

$$\mathcal{L}_{\text{GNN}} = -\log \frac{\exp(s_{\text{gnn}}(q, T^*)/\tau_t)}{\exp(s_{\text{gnn}}(q, T^*)/\tau_t) + \sum_j \exp(s_{\text{gnn}}(q, T_j^-)/\tau_t)}, \tag{15}$$

where T^* is the gold table, $\{T_j^-\}$ are hard negatives drawn from the dense retriever's top-K tables, and τ_t is a temperature. We update only $\{W_T, W_P, W_q, g, \text{HGT layers}\}$; the text encoder remains frozen. Figure 5 illustrates the construction.

Modality-gap calibration. Message passing over the bipartite graph aligns table and passage representations by propagating information through cell links. The gate $\sigma(g)$ makes this coupling explicit: when $\sigma(g) \approx 0$, the table representation remains close to its dense embedding; as the gate opens, neighborhood information contributes more strongly. A formal statement and proof of this gap-calibration property are given in Appendix C.

Fusion and reranking. We fuse the sparse, dense, and structural signals using reciprocal rank fusion [34], avoiding score-scale mismatch across heterogeneous scorers. A cross-encoder reranker [39] then rescores the top fused candidates and returns the highest-ranked table $\hat{T}$. Passage ranking is performed next within $\mathcal{N}(\hat{T})$ under an explicit budget (see below).

C. Structure-Conditioned Passage Discovery

Inputs and outputs. Given the retrieved table $\hat{T}$ and its cell-link neighborhood $\mathcal{N}(\hat{T})$, this stage returns a passage set $\mathcal{P} \subseteq \mathcal{P}_{\text{pool}}$ with $|\mathcal{P}| \le K_{\text{pas}}$.

Cell-link conditioned retrieval. As shown in Observations 6–7 (Section II-B), answer-bearing passages are often missed by similarity search but remain reachable through cell hyperlinks. We therefore impose a structural prior: given the retrieved table $\hat{T}$, we restrict passage retrieval to its cell-link neighborhood,

$$\mathcal{C}_{\text{link}}(\hat{T}) = \mathcal{N}(\hat{T}), \tag{16}$$

and rank candidates lexically:

$$\mathcal{P}_{\text{link}} = \text{top-}K_{\text{pas}}\{\text{BM25}(q, p) : p \in \mathcal{C}_{\text{link}}(\hat{T})\}. \tag{17}$$

This table-induced neighborhood acts as a high-recall structural filter: instead of searching the full corpus, SPARQ+ ranks only a small table-relevant candidate set. Let $\bar{d}$ denote the average number of outgoing hyperlinks per cell. Then $|\mathcal{N}(T)| \le \bar{d} \cdot |T.C|$, reducing the candidate pool from $M = |\mathcal{P}_{\text{pool}}|$ to $|\mathcal{N}(T)|$. Moreover, if the answer-bearing passage is linked from the gold table ($p^* \in \mathcal{N}(T^*)$) and Stage 1 retrieves that table ($\hat{T} = T^*$), then $p^* \in \mathcal{C}_{\text{link}}(\hat{T})$. A formal statement and proof of these properties are given in Appendix B.

Union fallback. This coverage guarantee is conditional: if Stage 1 misses the gold table or the answer-bearing passage is not hyperlinked, then $\mathcal{P}_{\text{link}}$ may exclude p^*. To improve robustness, SPARQ+ augments $\mathcal{P}_{\text{link}}$ with passages retrieved from the full corpus using dense retrieval and BM25. Passages in $\mathcal{P}_{\text{link}}$ are retained first; any remaining budget is filled by interleaving the open-pool rankings with duplicate removal. The resulting budgeted set is denoted by $\mathcal{P}$.

Passage budget. The budget K_{pas} instantiates the cost constraint in Eq. (8): increasing $|\mathcal{P}|$ may improve recall but enlarges the reader context. Since passage-answer recall exhibits diminishing returns [40], a small K_{pas} within $\mathcal{C}_{\text{link}}(\hat{T})$ often achieves near-saturated recall (Section V-E). Unlike learned passage selectors [41], [42], this stage remains training-free, while the structural prior mitigates noise by restricting retrieval to table-induced neighborhoods.

D. Passage-Aware Routing, Verification, and Recovery

From closed to open. We reuse the closed-setting router E, verifier Q, and rollback/fallback mechanisms (Section III).

The only change is that retrieved passages $\mathcal{P}$ are appended to the reasoning context together with the pruned subtable $T^{\mathcal{S}}$. We therefore extend routing and verification to use passage evidence, while keeping all other components unchanged.

Passage-aware routing. In the closed setting, operator fitness aggregates $P(\mathcal{S}_k \mid q, T)$ over candidate operator sets. In open setting, we condition this distribution on retrieved passages:

$$o_i.p = \sum_{\mathcal{S}_k:\, o_i \in \mathcal{S}_k} P(\mathcal{S}_k \mid q, T, \mathcal{P}). \tag{18}$$

This lets passage evidence steer operator selection: the router may bypass table extraction when the answer lies in passages, or select table operators when table filtering is also needed. When $\mathcal{P} = \emptyset$, Eq. (18) reduces to the closed-setting form. The router architecture is unchanged; only the classifier-based variant is retrained on open-domain samples with passages.

Passage-aware verification. In the closed setting, Q uses three alignments: query–subtable, query–execution, and subtable–execution (Section III-D). In the open setting, we extend it with query–passage and subtable–passage alignments:

$$\begin{aligned} Q' = \sigma\big(&\alpha f_\theta(q, T^{\mathcal{S}}) + \beta f_\theta(q, \mathsf{Context}(\mathcal{S})) \\ &+ \gamma f_\theta(T^{\mathcal{S}}, \mathsf{Context}(\mathcal{S})) + \delta f_\theta(q, \mathcal{P}) \\ &+ \epsilon f_\theta(T^{\mathcal{S}}, \mathcal{P})\big), \end{aligned} \tag{19}$$

where f_θ is the same cross-encoder as in Section III-D; by default, $\alpha = \beta = 1$, $\gamma = 0.2$, and $\delta = \epsilon = 0.5$. The first three terms recover Q, while the last two account for passage evidence. When $\mathcal{P} = \emptyset$, Q' reduces to Q. At inference, Q' is thresholded as a sufficiency test and triggers the same rollback/fallback mechanisms when needed.

Following Section III-D, the training extends contrastive learning from three modality pairs to five by adding $(q, \mathcal{P})$ and $(T^{\mathcal{S}}, \mathcal{P})$. The total loss is

$$\mathcal{L}_{\text{total}} = \alpha\mathcal{L}_{qT} + \beta\mathcal{L}_{qC} + \gamma\mathcal{L}_{TC} + \delta\mathcal{L}_{qP} + \epsilon\mathcal{L}_{TP}, \tag{20}$$

where each term uses the InfoNCE loss (Eq. (5)), and $\mathcal{L}_{qT}$, $\mathcal{L}_{qC}$, $\mathcal{L}_{TC}$, $\mathcal{L}_{qP}$ and $\mathcal{L}_{TP}$ correspond to $(q, T^{\mathcal{S}})$, $(q, \mathsf{Context}(\mathcal{S}))$, $(T^{\mathcal{S}}, \mathsf{Context}(\mathcal{S}))$, $(q, \mathcal{P})$ and $(T^{\mathcal{S}}, \mathcal{P})$, resp.

Theorem 1: *[Verifier training as an MI bound] Let the cross-encoder critic f_θ be trained with InfoNCE [35] over k negatives, and let $\mathcal{L}_{total} = \sum_{(X,Y)} w_{XY}\mathcal{L}_{XY}$, where $w_{XY} \in \{\alpha, \beta, \gamma, \delta, \epsilon\}$. Then $\sum_{(X,Y)} w_{XY} I(X;Y) \geq \left(\sum_{(X,Y)} w_{XY}\right) \log k - \mathcal{L}_{total}$.* □

Proof: It follows by applying the standard InfoNCE mutual-information bound [35], [43] to each modality pair and summing with non-negative weights. The full proof is in Appendix A. □

Thus, minimizing $\mathcal{L}_{\text{total}}$ maximizes a variational lower bound on the weighted mutual information among the query, evidence, and execution result.

Rollback and recovery. Rollback and fallback are inherited unchanged from Section III-D. When $Q' < \tau$, the system removes the lowest-fitness operator and re-evaluates the context; if the base case still fails, fallback invokes SQL reasoning over the full table. When $\mathcal{P} = \emptyset$, all passage-aware components vanish, recovering the original SPARQ pipeline.

Example 4: For the query "What nationality is the player with the most runs listed?" (Example 3), the router receives $(\hat{T}, \mathcal{P})$, where $\mathcal{P}$ contains the player's biography. It selects $\mathcal{S} = \{o^{\mathsf{col}}\}$ to retain only the "Player" and "Runs" columns before consulting the passage. If it instead over-selects $\{o^{\mathsf{col}}, o^{\mathsf{row}}\}$ and o^{row} removes the correct row (*e.g.,* "Tom Brown"), the verifier detects $Q' < \tau$ and rolls back by removing o^{row}. The restored context satisfies $Q' \geq \tau$, since the retained columns and linked passage together provide sufficient evidence.

V. Experiment

This section evaluates SPARQ+ under the unified minimal-sufficiency objective of Section IV-A, which the closed and open settings instantiate as two cases of the same evidence–sufficiency trade-off. We organize the evaluation from three perspectives. First, we examine *closed-setting performance*, where the corpus reduces to a single given table and SPARQ+ reduces to SPARQ, focusing on effectiveness and efficiency. Second, we evaluate *open-domain performance*, where both the relevant table and linked passages must be retrieved. Finally, we analyze *why* SPARQ+ works through component analysis, ablations, and retrieval–reasoning diagnosis, followed by end-to-end latency and runtime breakdown results.

A. Experimental Settings

Datasets. We evaluate SPARQ+ on 7 benchmarks: 5 closed-setting datasets and 2 open-domain table–text QA datasets. We report results in both the *closed* setting, where the target table is given, and the *open* setting, where both the relevant table and supporting passages must be retrieved. Closed-setting benchmarks evaluate the given-table reasoning capability, while open-setting benchmarks evaluate the full SPARQ+ pipeline, including table retrieval, passage discovery, and passage-aware reasoning. Due to space constraints, we briefly introduce each dataset below and defer detailed discussion to [27].

Closed-setting benchmarks. (a) *WikiTQ* [5]: short-answer compositional QA; (b) *FeTaQA* [3]: long-form answer generation; (c) *TabFact* [44]: fact verification; (d) *TableBench* [45]: heterogeneous enterprise tables; and (e) *NeedleInATable* (NIAT) [46]: dirty ultra-long spreadsheets.

Open-setting benchmarks. (f) *HybridQA* [24]: end-to-end retrieval and multi-hop reasoning over a retrieved table and its linked passages; and (g) *OTT-QA* [29]: large-corpus table retrieval and table–passage reasoning over millions of passages.

Baselines. We compare with representative training-based methods (*e.g.,* TaPas [47], TAPEX [48], TableGPT2 [49]), training-free methods (*e.g.,* DATER [11], ReAcTable [7], H-STAR [14]), and open-domain systems (*e.g.,* BM25, ColBERTv2 [31], HELIOS [17]). For fair comparison, we follow each baseline's standard evaluation protocol and cite best-reported results when artifacts are unavailable. Detailed configurations and additional baselines are in Appendix T [27].

Setup. Unless otherwise specified, we run offline experiments

TABLE IV: Closed-setting results on WikiTQ and TabFact (best in **bold**, second best underlined; /: not reported or irreproducible).

Method	WikiTQ	TabFact
Approaches requiring training		
TaPas (ACL′20)	48.8	83.9
TAPEX (ICLR′22)	57.5	86.7
LEVER (ICML′23)	62.9	/
Table-GPT (SIGMOD′24)	52.8	/
TABLEGPT2$_{7B}$	61.4	77.8
TABLEGPT2$_{70B}$	71.4	85.4
Approaches without training		
DATER(SIGIR′23)	65.9	85.6
TabSQLify (NAACL′24)	64.7	79.5
ReACTable (VLDB′24)	68.0	86.1
Chain-Of-Tables (ICLR′24)	67.3	86.6
HSTAR$_{gpt3.5}$ (NAACL′25)	69.56	86.5
HSTAR$_{gpt4}$	74.93	89.42
HSTAR$_{70B}$	75.76	89.23
SPARQ$_{4B}$	77.03	91.30
SPARQ$_{30B}$	**79.79**	**92.19**

TABLE V: Unified offline comparison on TabFact and WikiTQ using the same backend LLM. Latency is measured as seconds per query with 16 threads (TabSQLify is single-thread only). Full results on NIAT and TableBench are reported in Appendix U.

Method	Qwen3-4B				Qwen3-30B			
	TabFact		WikiTQ		TabFact		WikiTQ	
	Acc	Lat.	Acc	Lat.	Acc	Lat.	Acc	Lat.
ReAcTable	25.84	0.40	0.58	0.59	66.90	0.56	33.47	0.63
TabSQLify	6.23	3.14	63.58	5.08	45.06	2.96	72.26	4.46
H-STAR	60.33	1.66	8.98	2.06	88.24	2.26	70.33	2.48
Chain-of-Tables	17.74	1.20	49.15	2.02	83.75	2.12	58.31	1.96
SPARQ (Ours)	**91.30**	**0.31**	**77.03**	**0.49**	**92.19**	**0.33**	**79.79**	**0.55**

on 2 × RTX 4090 GPUs with vLLM [50] and report averages over three runs. For the closed setting, we use Qwen3-4B and Qwen3-30B (FP8) as offline backbones. For the open setting, we use a unified 35B reader for fair retriever comparison, set $K_{\text{pas}} = 15$, and rerank the top-20 fused table candidates; see Appendix W [27] for more detail of the setup.

Metrics. We report denotation accuracy for WikiTQ, classification accuracy for TabFact, EM for NIAT, ROUGE-L for FeTaQA and TableBench, and EM/F1 for open-domain HybridQA and OTT-QA. Detailed hyperparameter and sensitivity analyses are deferred to Appendix X [27].

B. Closed-Setting Performance

Effectiveness. Tables IV and V show that, in the closed setting where the target table is given, SPARQ+ achieves strong effectiveness. In this setting, SPARQ+ reduces to the original SPARQ framework [26].

Quantitative gains. On WikiTQ, SPARQ+ achieves 79.79% accuracy, improving over the strongest reported training-free baseline by 4.0 pp (Table IV). Under a unified offline backend, it also achieves 79.79% accuracy (Table V). On TabFact, SPARQ+ achieves 92.19% accuracy, outperforming the strongest training-free baseline by 2.8 pp (Table IV); under the unified backend, it again reaches 92.19% accuracy (Table V).

Beyond WikiTQ and TabFact, the same trend holds on FeTaQA, TableBench, and NIAT (Appendix T), which stress long-form generation, heterogeneous enterprise schemas, and dirty ultra-long spreadsheets, respectively. Since the table is given, this setting isolates the SPARQ core from open-domain retrieval errors in Stages 1–2. SPARQ+ remains robust because (a) the router selects operators that match the dominant failure mode (*e.g.,* symbolic execution for aggregation or arithmetic, and semantic filtering for entity-centric queries), and (b) the verifier rejects overly pruned contexts that would otherwise force the LLM to guess from incomplete evidence. This effect is especially pronounced for small offline models, where long reasoning chains can amplify early mistakes.

TABLE VI: Open-domain end-to-end results. Latency = index + inference, seconds per query on average.

Method	HybridQA (n=3,466)			OTT-QA (n=2,214)		
	EM	F1	Latency	EM	F1	Latency
SPARQ [26]	0.260	0.307	0.230	0.237	0.283	0.415
BM25	0.222	0.285	0.203	0.258	0.295	0.256
ColBERTv2 [31]	0.240	0.307	0.337	0.240	0.294	4.869
N2N-GQA [51]	0.118	0.149	1.870	0.111	0.162	1.953
CARP [52]	0.246	0.316	2.301	0.332	0.386	2.553
CORE [30]	0.375	0.423	3.093	0.490	0.569	3.220
COS [53]	—	—	—	0.504	0.566	4.155
HELIOS [17]	0.495	0.532	23.73	0.589	0.657	28.910
SPARQ+ **(ours)**	**0.508**	**0.549**	0.776	**0.676**	**0.732**	0.973

Efficiency. Adaptive routing avoids expensive full-table prompting by (a) answering easy queries directly and (b) invoking symbolic operators only when they are likely to succeed. Under a unified offline backend, SPARQ+ achieves 0.55 s/query on WikiTQ and 0.33 s/query on TabFact, compared with 4.46 s for TabSQLify and 2.26 s for H-STAR, respectively (Table V). The scheduler further overlaps CPU-bound execution (*e.g.,* SQL) with GPU-bound LLM inference, reducing end-to-end latency while preserving accuracy.

C. Open-Domain Performance

Having established that SPARQ+ matches SPARQ in the closed regime, we now turn to the open regime, which the closed setting does not cover. We evaluate SPARQ+ in the open-domain setting on OTT-QA (2,214 queries, 5.96M passages) and HybridQA (3,466 queries, 75K passages, with gold metadata removed). Unlike the closed setting, both the relevant table and supporting passages must be retrieved before reasoning.

Effectiveness. As shown in Table VI, SPARQ+ achieves the highest EM/F1 among the compared offline/open-domain baselines on both datasets: 0.508 EM/0.549 F1 on HybridQA and 0.676 EM/0.732 F1 on OTT-QA. The improvement over directly applying the closed-setting pipeline in the open setting confirms that open-domain table–text QA requires more than table reasoning: it also depends on accurate table retrieval, linked-passage discovery, and passage-aware reasoning.

Quantitative gains. On OTT-QA, SPARQ+ improves EM by 8.7 pp over HELIOS and 17.2 pp over COS; on HybridQA, it improves EM by 1.3 pp over HELIOS. It also substantially outperforms single-shot retrievers such as BM25 and ColBERTv2 on both datasets, indicating that the gains come from better evidence assembly and reasoning rather than from retrieval

TABLE VII: Adaptivity across table scales (WikiTQ; left) and robustness under long contexts (NIAT; right).

Method	Small	Med.	Large
DATER	62.5	42.3	34.6
CoT	68.1	52.3	44.9
TabSQL	68.2	57.9	52.3
H-STAR	71.6	65.2	64.8
SPARQ+$_{4B}$	79.0	74.8	70.3
SPARQ+$_{30B}$	**82.3**	**77.9**	**74.7**

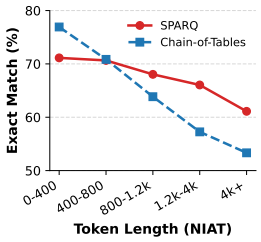


TABLE VIII: Ablation study of the accuracy performance for SPARQ+$_{4B}$ on TabFact and WikiTQ datasets.

Method	TabFact	WikiTQ
SPARQ+$_{4B}$	**91.30**	**77.04**
w. Training-Free Router E_{4B}	86.86	75.53
w/o Verification Model Q	87.20	74.06
w/o Table Extraction	87.10	74.38
w/o SQL Operator	87.70	74.24
w/o Retrieval Operator	87.55	76.26

alone. The margins are larger on OTT-QA, where large-corpus table retrieval is the dominant bottleneck.

Efficiency. SPARQ+ remains below 1 s/query on average, with 0.973 s on OTT-QA and 0.776 s on HybridQA (Table VI). It is consistently faster than retriever-heavy baselines (*e.g.,* ColBERTv2: 4.869 s on OTT-QA) and dramatically faster than iterative pipelines (*e.g.,* HELIOS: 28.910 s on OTT-QA and 23.73 s on HybridQA), while still achieving the highest EM/F1. This efficiency comes from offline indexing, structure-conditioned retrieval, and compact evidence windows for the reader, which shift the runtime bottleneck away from corpus-wide passage search and iterative reasoning.

D. Closed-Setting Component Analysis and Ablation

Having reported performance in both regimes, we now analyze *why* SPARQ+ works, starting from the closed-setting core shared by both. This subsection analyzes the adaptivity and operator-level contributions of the closed-setting core of SPARQ+, where it reduces to SPARQ.

Adaptivity across table scales. We analyze two representative closed-setting scenarios.

WikiTQ: table scale. As shown in Table VII (left), SPARQ+ outperforms baselines across tables, with stable gains rather than gains concentrated in a single regime; *e.g.,* SPARQ+$_{4B}$ achieves 79.0/75.1/70.3 on Small/Medium/Large tables, while SPARQ+$_{30B}$ further improves to 82.3/77.9/74.7. This stability suggests that the main benefit comes from selecting *minimal-but-sufficient* evidence: the router avoids over-reasoning on small tables and applies stronger pruning or symbolic execution on large tables, while the verifier prevents critical rows or columns from being discarded.

NIAT: long-context robustness. As shown in Table VII (right), SPARQ+ remains robust in long-context settings where baselines suffer a sharp reasoning cliff. NIAT spreadsheets contain noisy headers, heterogeneous layouts, duplicated entities, and non-tabular artifacts, making direct full-context prompting brittle for small offline LLMs. SPARQ+ mitigates this by routing to structure-aware operators and using verification to reject over-aggressive compression, yielding smoother degradation instead of catastrophic failure.

TABLE IX: Runtime and throughput breakdown on WikiTQ with Qwen3-4B (single thread; seconds and tokens/s).

Method	Avg. pred.	Avg. exec.	Avg. I/O throughput	Avg. latency
TabSQLify	1.96	1.49	2870 / 125	3.45
ReAcTable	3.52	0.07	**11601** / 114	3.59
H-STAR	6.53	8.66	1709 / 179	15.19
SPARQ$_{4B}$	**0.7612**	**0.0860**	5260 / **1184**	**0.8472**

Ablation. Table VIII ablates key components of the closed-setting core. The full system achieves 91.30 on TabFact and 77.04 on WikiTQ; removing any major component degrades performance, showing that the gains arise from the interaction of routing, operator execution, and verification.

Verifier. Removing verifier Q causes the largest drop (91.30→87.20 on TabFact; 77.04→74.06 on WikiTQ), confirming the importance of sufficiency verification. Without Q, it may over-compress the table and discard necessary evidence.

Router. Replacing the learned router with the training-free heuristic E_{4B} hurts accuracy (91.30→86.86; 77.04→75.53). This suggests that operator selection is instance-dependent and cannot be reliably handled by simple rules (*e.g.,* multi-hop questions or tables with distractor columns).

Retrieval operator. Here, the retrieval operator denotes in-table semantic retrieval in the closed setting, not open-domain table or passage retrieval. Removing it degrades both datasets (91.30→87.55 on TabFact; 77.04→76.26 on WikiTQ), showing that semantic evidence selection complements symbolic and structural operators. The larger drop on TabFact suggests that missing key evidence is especially harmful for statement verification, while the smaller WikiTQ drop indicates that retrieval alone is insufficient without verification.

SQL operator and table extraction. Removing SQL execution (91.30→87.70; 77.04→74.24) validates symbolic computation for aggregation, comparison, and multi-condition filtering. Removing table extraction (91.30→87.10; 77.04→74.38) shows that clean, canonicalized table representations are also essential, since extraction errors propagate to downstream operators.

We defer cross-domain generalization, operator-sequence distribution, and detailed analyses of table extraction and SQL/Python execution to Appendix L–M.

Overall, the ablations show that reliable closed-setting performance requires sufficiency verification, learned routing, and a combination of semantic and symbolic operators.

Efficiency micro-breakdown. Table IX reports a single-thread runtime breakdown, including prediction time, execution time, and throughput. Compared with baselines, SPARQ reduces prediction time through fewer LLM calls, reduces execution time through batched non-blocking tool execution, and maintains high output throughput via batch serving.

E. Open-Setting Retrieval Ablation and Diagnosis

Progressive retrieval ablation. Table X decomposes the open-setting gains on HybridQA along three axes: Stage-1 table

TABLE X: HybridQA progressive ablation (n=3,466). Table R@1 = Stage-1 table retrieval Recall@1. Psg R@K = passage answer-recall (gold answer appears in top-K passages). Tokens = average reader input tokens per query.

Configuration	EM	F1	Table R@1	Psg R@1	Psg R@5	Tokens
Base (open-pool, K=30)	0.362	0.399	0.609	0.141	0.306	8,120
+ cell-link	0.461	0.503	0.609	0.189	**0.422**	5,724
+ union fallback	0.466	0.509	0.609	0.189	0.414	7,825
+ K_{pas}=80	0.492	0.534	0.609	**0.192**	0.415	13,054
+ Stage-1 reranker	**0.508**	**0.549**	**0.625**	**0.192**	0.415	13,054

TABLE XI: Fixed-evidence reasoning comparison on OTT-QA (EM). CoT = operator-routed chain-of-thought reader; Direct = single-pass short-answer reader on the same 35B model; S1✓/S1× denote whether Stage 1 retrieves the correct table.

Evidence	Reader	All	S1✓	S1×
SPARQ+ Stage 1–2	CoT (35B)	**0.676**	**0.741**	0.170
SPARQ+ Stage 1–2	Direct (35B)	0.621	0.676	**0.194**
HELIOS retrieval	35B	0.590	—	—
HELIOS retrieval	FiE	0.589	—	—

ranking (Table R@1), passage answer recall (Psg R@K), and reader token budget. A key pattern is that higher EM/F1 often comes with less reader context, showing that SPARQ+ improves by assembling more sufficient evidence rather than simply feeding more evidence. Moreover, we find the following.

(1) Cell-link yields the largest gain while reducing context. Adding cell-link retrieval increases EM/F1 from 0.362/0.399 to 0.461/0.503, while reducing the average token budget from 8,120 to 5,724. This improvement coincides with a large increase in passage answer recall (Psg R@5: 0.306→0.422), suggesting that constraining passage retrieval to table-linked neighborhoods is more effective than similarity search.

(2) Union fallback improves robustness at a token cost. Union fallback slightly improves EM/F1 from 0.461/0.503 to 0.466/0.509, but increases tokens from 5,724 to 7,825. This matches its role as a safety net: when links are missing or noisy, it recovers recall by relaxing the structural constraint, at the cost of a wider reader context.

(3) When budget expansion saturates, reranking can still boost performance. Increasing K_{pas} to 80 substantially increases tokens from 7,825 to 13,054, but yields only a modest EM/F1 gain from 0.466/0.509 to 0.492/0.534. This illustrates diminishing returns from simply expanding the passage budget. In contrast, reranking improves Table R@1 from 0.609 to 0.625 and achieves the best EM/F1, 0.508/0.549, without changing passage recall or token budget. This indicates that once passage neighborhoods are saturated, table-ranking precision becomes the main bottleneck.

Retrieval–reasoning diagnosis. We next diagnose how evidence quality affects reasoning.

Evidence quality gates reasoning. Table XI holds the evidence fixed and varies only the reader. When Stage 1 retrieves the correct table (S1✓), operator-routed reasoning outperforms direct CoT reasoning (0.741 vs. 0.676), indicating that structured routing helps the reader exploit aligned table–passage evidence. However, when Stage 1 misses the table (S1×), long-form reasoning becomes brittle (0.170 vs. 0.194): with an incorrect table, multi-step rationales can amplify spurious evidence into confident but wrong answers. These results reinforce our emphasis on the Stage-1 precision and on providing minimal-but-sufficient reasoning context.

TABLE XII: Error cascade decomposition of SPARQ+. Table miss = Stage-1 retrieves a wrong table. Evid. miss = table correct but the answer-bearing passage is absent from the assembled evidence. Reader fail = evidence complete but the answer is wrong.

	Correct	Table miss	Evid. miss	Reader fail
HybridQA (n=3,466)	45.0%	37.5%	0.6%	16.8%
OTT-QA (n=2,214)	65.6%	11.4%	0.4%	22.6%

Error cascade. Table XII shows that errors mainly derive from the Stage-1 table misses (37.5% on HybridQA; 11.4% on OTT-QA), whereas evidence misses after retrieving the correct table are rare ($\leq$0.6%). This leads to: (1) once the correct table is retrieved, the cell-link neighborhood largely covers the answer-bearing passages, so increasing the passage budget yields limited additional recall; and (2) the remaining headroom lies mainly in improving Stage-1 ranking/fusion and strengthening reader-side synthesis over already sufficient evidence. Consequently, we focus the open-setting ablations on retrieval components, while the router and verifier are ablated in the closed regime (Section V-D), where their contribution can be isolated from retrieval noise. Additional diagnostics are deferred to Appendix W in full version [27].

VI. Related Work

We classify the related work into the following categories.

Open-Domain Table–Text Retrieval & QA. HybridQA [24] studies QA over a given table and its linked passages, while OTT-QA [29] extends the problem to open-domain retrieval. Subsequent work improves retrieval through dense retrievers [22], reasoning chains (CORE [30], CARP [52]), modular retrieval (COS [53]), late interaction [31], multi-granularity retrieval [54], hybrid retrieval frameworks [17], compact subgraph retrieval [18], [51], and graph-based reasoning over a given table [55]. These methods primarily improve retrieval and rely on a generic reader for reasoning. In contrast, SPARQ+ jointly optimizes evidence discovery and operator-level reasoning under a unified sufficiency objective, constrains passage retrieval through the cell-link graph, and targets fully offline deployment.

Large Language Models for TQA. Cloud-hosted LLMs (*e.g.,* GPT [56] and PaLM [57]) offer strong reasoning but raise latency, privacy, and deployment concerns, while offline models (*e.g.,* Qwen [28] and LLaMA [58]) support local deployment but are less robust on complex reasoning. Even table-specialized LLMs [8], [49] remain limited by numerical reasoning, hallucinations, and relational modeling [8], [59], [60]. SPARQ instead boosts offline reasoning via operator routing and sufficiency verification rather than larger models.

Natural Language to SQL. NL2SQL has evolved from grammar-based methods [61], [62] to neural models [63]–[65] and LLM-based generation [56], [66]–[68], supported by benchmarks such as Spider [69], CoSQL [70], WikiSQL [71], and BIRD [72]. Recent TQA systems integrate iterative SQL reasoning [7], [13], [68]. Unlike these, SPARQ treats SQL as one operator within a modular reasoning framework, allowing fallback without relying entirely on SQL generation.

Chain-of-Thought Reasoning. CoT and multi-agent reasoning improve multi-step reasoning through intermediate decomposition [9], [73]–[75], but often incur excessive inference cost and overthinking [76]–[79]. Representative TQA methods include DATER [11], Chain-of-Table [12], H-STAR [14], RoT [15], and AutoTQA [9], [74]. In contrast, SPARQ performs lightweight operator routing with sufficiency verification, achieving efficient and stable reasoning on offline models.

Evidence Sufficiency and Context Selection. Prior work studies compact evidence selection through complementary retrieval [41], redundancy-aware selection [42], information-bottleneck filtering [80], and sufficient-context diagnosis [81]. These methods focus on flat text and operate only during retrieval. In contrast, the verifier Q' in SPARQ+ jointly governs retrieval and reasoning over heterogeneous table–text evidence, supporting rollback and adaptive passage budgeting.

VII. Conclusion

We presented SPARQ+, an offline framework for open-domain table-text QA that addresses the privacy, latency, and cost limitations of online systems. SPARQ+ extends SPARQ's minimal-sufficiency principle from operator selection to evidence discovery: it combines gate-controlled heterogeneous GNN table retrieval, cell-link constrained passage discovery, and passage-aware reasoning under a unified cost–sufficiency objective grounded in the information bottleneck principle. By coupling an adaptive router with a mutual-information verifier, SPARQ+ selects efficient reasoning paths while enforcing evidence sufficiency. Across extensive experiments, we demonstrate the effectiveness and efficiency of SPARQ+.

Future work include: (a) improving table ranking to reduce retrieval errors, (b) extending structural priors beyond cell hyperlinks to cover cases where answer-bearing passages fall outside link neighborhoods, and (c) using overlong generations as a runtime signal to trigger re-retrieval.

References

[1] N. Jin, J. Siebert *et al.*, "A survey on table question answering: recent advances," in *CCKS*, 2022, pp. 174–186.

[2] X. Zhang, D. Wang *et al.*, "A survey of table reasoning with large language models," *Front. Comput. Sci.*, vol. 19, no. 9, p. 199348, 2025.

[3] L. Nan, C. Hsieh *et al.*, "FeTaQA: Free-form table question answering," *TACL*, vol. 10, pp. 35–49, 2022.

[4] X. Deng, H. Sun *et al.*, "Turl: Table understanding through representation learning," *ACM SIGMOD Rec.*, vol. 51, no. 1, pp. 33–40, 2022.

[5] P. Pasupat and P. Liang, "Compositional semantic parsing on semi-structured tables," in *ACL*, 2015, pp. 1470–1480.

[6] Y. Chung, G. T. Kakkar *et al.*, "Is long context all you need? leveraging LLM's extended context for NL2SQL," *PVLDB*, vol. 18, no. 8, pp. 2735–2747, 2025.

[7] Y. Zhang, J. Henkel *et al.*, "ReAcTable: Enhancing react for table question answering," *PVLDB*, vol. 17, no. 8, pp. 1981–1994, 2024.

[8] P. Li, Y. He *et al.*, "Table-gpt: Table fine-tuned gpt for diverse table tasks," *PACMMOD*, vol. 2, no. 3, pp. 1–28, 2024.

[9] J.-P. Zhu, P. Cai *et al.*, "Autotqa: Towards autonomous tabular question answering through multi-agent large language models," *PVLDB*, vol. 17, no. 12, pp. 3920–3933, 2024.

[10] S.-A. Chen, L. Miculicich *et al.*, "TableRAG: Million-token table understanding with language models," in *NeurIPS*, 2024.

[11] Y. Ye, B. Hui *et al.*, "Large language models are versatile decomposers: Decomposing evidence and questions for table-based reasoning," in *SIGIR*, 2023, pp. 174–184.

[12] Z. Wang, H. Zhang *et al.*, "Chain-of-table: Evolving tables in the reasoning chain for table understanding," in *ICLR*, 2024.

[13] M. M. H. Nahid and D. Rafiei, "TabSQLify: Enhancing reasoning capabilities of LLMs through table decomposition," in *NAACL*, 2024, pp. 5725–5737.

[14] N. Abhyankar, V. Gupta *et al.*, "H-STAR: LLM-driven hybrid SQL-text adaptive reasoning on tables," in *NAACL*, 2025, pp. 8841–8863.

[15] X. Zhang, D. Wang *et al.*, "RoT: Enhancing table reasoning with iterative row-wise traversals," in *EMNLP*, 2025, pp. 559–579.

[16] Y. Jiang, F. Wei *et al.*, "Accurate table question answering with accessible LLMs," in *ICDE*, 2026.

[17] S. Park, J. Yun *et al.*, "HELIOS: Harmonizing early fusion, late fusion, and LLM reasoning for multi-granular table-text retrieval," in *ACL*, 2025, pp. 32 424–32 444.

[18] V. Kumar, J. Sen *et al.*, "Graph representation of tables+text and compact subgraph retrieval for QA tasks," in *ECIR*, vol. 15572, 2025, pp. 164–180.

[19] Z. Wan, X. Wang *et al.*, "Efficient large language models: A survey," *TMLR*, 2024.

[20] L. Chen, M. Zaharia, and J. Zou, "Frugalgpt: How to use large language models while reducing cost and improving performance," *TMLR*, 2024.

[21] W. Liang, Y. Zhang *et al.*, "Mind the gap: Understanding the modality gap in multi-modal contrastive representation learning," in *NeurIPS*, 2022.

[22] J. Herzig, T. Müller *et al.*, "Open domain question answering over tables via dense retrieval," in *NAACL*, 2021, pp. 512–519.

[23] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends® in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[24] W. Chen, H. Zha *et al.*, "HybridQA: A dataset of multi-hop question answering over tabular and textual data," in *Findings of EMNLP*, 2020, pp. 1026–1036.

[25] W. Xiong, X. L. Li *et al.*, "Answering complex open-domain questions with multi-hop dense retrieval," in *ICLR*, 2021.

[26] Y. Liu, M. Yan *et al.*, "SPARQ: A cost-efficient framework for offline table question answering via adaptive routing," in *ICDE*, 2026.

[27] SPARQ Authors, "SPARQ: Source code and full experimental results," Online: https://github.com/authurlord/SPARQ_Extend_Release, 2026.

[28] Qwen Team, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.

[29] W. Chen, M.-W. Chang *et al.*, "Open question answering over tables and text," in *ICLR*, 2021.

[30] K. Ma, H. Cheng *et al.*, "Open-domain question answering via chain of reasoning over heterogeneous knowledge," in *Findings of EMNLP*, 2022, pp. 5360–5374.

[31] K. Santhanam, O. Khattab *et al.*, "ColBERTv2: Effective and efficient retrieval via lightweight late interaction," in *NAACL*, 2022, pp. 3715–3734.

[32] X. Ma, Y. Gong *et al.*, "Query rewriting in retrieval-augmented large language models," in *EMNLP*, 2023.

[33] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *EMNLP-IJCNLP*, 2019, pp. 3982–3992.

[34] G. V. Cormack, C. L. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms condorcet and individual rank learning methods," in *SIGIR*, 2009, pp. 758–759.

[35] A. van den Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," *arXiv preprint arXiv:1807.03748*, 2018.

[36] N. Tishby, F. C. Pereira, and W. Bialek, "The information bottleneck method," *arXiv preprint physics/0004057*, 2000, originally presented at the 37th Annual Allerton Conference on Communication, Control, and Computing, 1999.

[37] N. Tishby and N. Zaslavsky, "Deep learning and the information bottleneck principle," in *IEEE ITW*, 2015, pp. 1–5.

[38] Z. Hu, Y. Dong *et al.*, "Heterogeneous graph transformer," in *WWW*, 2020, pp. 2704–2710.

[39] J. Chen, S. Xiao *et al.*, "BGE M3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," in *Findings of ACL*, 2024, pp. 2318–2335.

[40] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher, "An analysis of approximations for maximizing submodular set functions—I," *Math. Program.*, vol. 14, pp. 265–294, 1978.

[41] X. Mou, M. Yu *et al.*, "Complementary evidence identification in open-domain question answering," in *EACL*, 2021, pp. 2720–2726.

[42] C. Peng, B. Wang *et al.*, "AdaGReS: Adaptive greedy context selection via redundancy-aware scoring for token-budgeted RAG," *arXiv preprint arXiv:2512.25052*, 2025.

[43] B. Poole, S. Ozair *et al.*, "On variational bounds of mutual information," in *ICML*, vol. 97, 2019, pp. 5171–5180.
[44] W. Chen, H. Wang *et al.*, "Tabfact: A large-scale dataset for table-based fact verification," in *ICLR*, 2020.
[45] X. Wu, J. Yang *et al.*, "TableBench: A comprehensive and complex benchmark for table question answering," in *AAAI*, vol. 39, no. 24, 2025, pp. 25 497–25 506.
[46] L. Wang, M. Zheng *et al.*, "NeedleInATable: Exploring long-context capability of large language models towards long-structured tables," in *NeurIPS*, 2025.
[47] J. Herzig, P. K. Nowak *et al.*, "TaPas: Weakly supervised table parsing via pre-training," in *ACL*, 2020, pp. 4320–4333.
[48] Q. Liu, B. Chen *et al.*, "Tapex: Table pre-training via learning a neural sql executor," in *ICLR*, 2022.
[49] A. Su, A. Wang *et al.*, "TableGPT2: A large multimodal model with tabular data integration," *arXiv preprint arXiv:2411.02059*, 2024.
[50] W. Kwon, Z. Li *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *SOSP*, 2023, pp. 611–626.
[51] M. Sharafath, A. Annamalai *et al.*, "N2N-GQA: Noise-to-narrative for graph-based table-text question answering using LLMs," *arXiv preprint arXiv:2601.06603*, 2026.
[52] W. Zhong, J. Huang *et al.*, "Reasoning over hybrid chain for table-and-text open domain question answering," in *IJCAI*, 2022, pp. 4531–4537.
[53] K. Ma, H. Cheng *et al.*, "Chain-of-skills: A configurable model for open-domain question answering," in *ACL*, 2023, pp. 1599–1618.
[54] Y. Wang, J. Bao *et al.*, "MuGER2: Multi-granularity evidence retrieval and reasoning for hybrid question answering," in *Findings of EMNLP*, 2022, pp. 6687–6697.
[55] A. Agarwal, C. Devaguptapu, and G. S, "Hybrid graphs for table-and-text based question answering using LLMs," in *NAACL*, 2025, pp. 858–875.
[56] T. Brown, B. Mann *et al.*, "Language models are few-shot learners," *NeurIPS*, vol. 33, pp. 1877–1901, 2020.
[57] R. Anil, A. M. Dai *et al.*, "Palm 2 technical report," *arXiv preprint arXiv:2305.10403*, 2023.
[58] A. Grattafiori, A. Dubey *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.
[59] J. Ahn, R. Verma *et al.*, "Large language models for mathematical reasoning: Progresses and challenges," in *EACL SRW*, 2024, pp. 225–237.
[60] P. Shojaee, I. Mirzadeh *et al.*, "The illusion of thinking: Understanding the strengths and limitations of reasoning models via the lens of problem complexity," *arXiv preprint arXiv:2506.06941*, 2025.
[61] T. Gvero and V. Kuncak, "Synthesizing java expressions from free-form queries," in *OOPSLA*, 2015, pp. 416–432.
[62] C. Quirk, R. Mooney, and M. Galley, "Language to code: Learning semantic parsers for if-this-then-that recipes," in *ACL*, 2015, pp. 878–888.
[63] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *NeurIPS*, 2014.
[64] X. Gu, H. Zhang, and S. Kim, "Deep code search," in *ICSE*, 2018, pp. 933–944.
[65] Z. Feng, D. Guo *et al.*, "Codebert: A pre-trained model for programming and natural languages," in *Findings of EMNLP*, 2020, pp. 1536–1547.
[66] M. Chen, J. Tworek *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
[67] T. Scholak, N. Schucher, and D. Bahdanau, "Picard: Parsing incrementally for constrained auto-regressive decoding from language models," in *EMNLP*, 2021, pp. 9895–9901.
[68] A. Ni, S. Iyer *et al.*, "Lever: Learning to verify language-to-code generation with execution," in *ICML*, 2023, pp. 26 106–26 128.
[69] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Radev, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task," in *EMNLP*, 2018, pp. 3911–3921.
[70] T. Yu *et al.*, "Cosql: A conversational text-to-sql challenge towards cross-domain natural language interfaces to databases," in *EMNLP*, 2019, pp. 1962–1979.
[71] V. Zhong, C. Xiong, and R. Socher, "Seq2sql: Generating structured queries from natural language using reinforcement learning," *arXiv preprint arXiv:1709.00103*, 2017.
[72] J. Li, B. Hui *et al.*, "Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls," *NeurIPS*, vol. 36, pp. 42 330–42 357, 2023.
[73] J. Wei, X. Wang *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," in *NeurIPS*, vol. 35, 2022, pp. 24 824–24 837.
[74] J. Zhu, P. Cai *et al.*, "Unitqa: A unified automated tabular question answering system with multi-agent large language models," in *SIGMOD*, 2025, pp. 279–282.
[75] Q. Wu, G. Bansal *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," in *COLM*, 2024.
[76] Y. Xu, X. Guo *et al.*, "SoftCoT++: Test-time scaling with soft chain-of-thought reasoning," *arXiv preprint arXiv:2505.11484*, 2025.
[77] Y. Ge, S. Liu *et al.*, "Innate reasoning is not enough: In-context learning enhances reasoning large language models with less overthinking," *arXiv preprint arXiv:2503.19602*, 2025.
[78] Z. Wei, L. Pang *et al.*, "The evolution of thought: Tracking LLM overthinking via reasoning dynamics analysis," in *ACL*, 2026, pp. 26 905–26 920.
[79] G. Xiao, D. He *et al.*, "CENTS: A flexible and cost-effective framework for LLM-based table understanding," *PVLDB*, vol. 18, no. 11, pp. 4574–4587, 2025.
[80] K. Zhu, X. Feng *et al.*, "An information bottleneck perspective for effective noise filtering on retrieval-augmented generation," in *ACL*, 2024, pp. 1044–1069.
[81] H. Joren, J. Zhang *et al.*, "Sufficient context: A new lens on retrieval augmented generation systems," in *ICLR*, 2025.
[82] W. Yeo, K. Kim *et al.*, "UniversalRAG: Retrieval-augmented generation over corpora of diverse modalities and granularities," in *ACL*, 2026, pp. 3843–3871.
[83] Qwen Team, "Qwen3-4B-Instruct-2507," Hugging Face model card, 2025, online: https://huggingface.co/Qwen/Qwen3-4B-Instruct-2507.
[84] ——, "Qwen3-30B-A3B-Instruct-2507-FP8," Hugging Face model card, 2025, online: https://huggingface.co/Qwen/Qwen3-30B-A3B-Instruct-2507-FP8.
[85] Z. Ye, L. Chen *et al.*, "Flashinfer: Efficient and customizable attention engine for LLM inference serving," in *MLSys*, 2025.
[86] A. Kuzmin, M. Van Baalen *et al.*, "Fp8 quantization: The power of the exponent," *NeurIPS*, vol. 35, pp. 14 651–14 662, 2022.
[87] Qwen Team, "Qwen3-Max: Just scale it," Official Qwen blog, 2025, online: https://qwen.ai/blog?id=qwen3-max.

Mengyi Yan received the PhD degree from the School of Computer Science and Engineering, Beihang University in 2025. He is currently an assistant professor in School of Artificial Intelligence, Shandong University. His research interests include Data Quality, Machine Learning and Database.

Yang Liu is currently working toward his PhD degree at the School of Computer Science and Engineering, Beihang University, China. His research interests include graph processing, distributed system and LLM inference optimization.

Weilong Ren received his Ph.D. degree in computer science from Kent State University in 2021. He is currently a research scientist at the Shenzhen Institute of Computing Sciences, Shenzhen, China. His research interests include data management for machine learning, data quality, and query processing and optimization.

Yutong Ye received his Ph.D. in Software Engineering from East China Normal University, China in 2025. He is currently a Postdoctoral Researcher at the School of Computer Science, Beihang University, China. His research lies at the intersection of graph data management and artificial intelligence, with a focus on graph query processing, graph representation learning, and learning-based data systems.

Haoyi Zhou is an Associate Professor with Beihang University, Beijing, China. He received the Ph.D. degree from Beihang University in 2021 and was a visiting scholar at Rutgers University. His research interests include time-series forecasting and AI-enabled scientific computing. He was selected for the Young Elite Scientists Sponsorship Program by CAST and was named among the 2024 Most Cited Chinese Researchers.

Jianxin Li received his Ph.D. degree from the School of Computer Science and Engineering at Beihang University, China, in 2007. He is currently a professor with the School of Computer Science and Engineering, Beihang University, and a senior researcher with Beijing Advanced Innovation Center for Big Data and Brain Computing. His current research interests consist of social networks, machine learning, Big Data, and trustworthy computing.

Zhumin Chen received the Ph.D. degree from Shandong University, China, in 2008. He is currently a Professor and Ph.D. Supervisor with the School of Computer Science and Technology, Shandong University, Qingdao, China. He is a Senior Member of the China Computer Federation (CCF). His research interests include natural language processing, big data processing, and recommendation systems.

TABLE XIII: Notation summary. ⋆: new in the open-domain extension.

Symbol	Description	
Closed setting (Section III-A)		
$\mathcal{O}$, o^i	Operator pool; single operator	
$\mathcal{S}$	Selected operator set $\subseteq \mathcal{O}$	
$T^{\mathcal{S}}$, $\mathsf{Context}(\mathcal{S})$	Subtable; LLM intermediate result	
τ	Sufficiency threshold	
Open-domain extension (Section IV)		
$\mathcal{T}_{\text{pool}}$, $\mathcal{P}_{\text{pool}}$	Table / passage corpus	⋆
$\mathcal{G}$, $\mathcal{V}_T \cup \mathcal{V}_P$, $\mathcal{E}_G$	Bipartite graph; node / edge sets	⋆
$\mathcal{N}(T)$, $\bar{d}$	Cell-link neighborhood; avg. out-degree	⋆
$\mathcal{P}$, K_{pas}	Selected passages; passage budget	⋆
Stage 1 (Section IV-B): Table retrieval		
s_{bm25}, s_{dense}, s_{gnn}	Three retrieval scores	⋆
$\tilde{\mathbf{h}}_T$, g, τ_t	Gate-fused embedding; gate; InfoNCE temp.	⋆
$\hat{T}$	Retrieved top-1 table	⋆
Stage 2 (Section IV-C): Passage discovery		
$\mathcal{C}_{\text{link}}$, $\mathcal{C}_{\text{open}}$	Cell-link / open-pool candidates	⋆
Stage 3 (Section IV-D): Reasoning		
E, $o_i.p$	Router; operator fitness probability	
Q, Q'	Verifier (closed / open-domain)	⋆
α, β, γ	Verifier weights (base)	
δ, ϵ	Passage-aware weights	⋆

APPENDIX

A. Proof of Theorem 1 (Verifier as MI Bound)

Proof: For a modality pair (X, Y), the InfoNCE loss with k negative samples satisfies [35], [43]:

$$I(X;Y) \;\geq\; \log k - \mathcal{L}_{XY}(f_\theta).$$

This is the standard result of van den Oord et al. (2018), proved via a variational bound on the density ratio estimated by the critic f_θ.

The verifier is trained over the pair set $\mathcal{M} = \{(q, T^{\mathcal{S}}), (q, \mathsf{Context}(\mathcal{S})), (T^{\mathcal{S}}, \mathsf{Context}(\mathcal{S})), (q, \mathcal{P}), (T^{\mathcal{S}}, \mathcal{P})\}$ with non-negative weights $w_{XY} \in \{\alpha, \beta, \gamma, \delta, \epsilon\}$ (Eq. (19)). Summing the per-pair bounds with these weights:

$$\sum_{(X,Y)\in\mathcal{M}} w_{XY}\, I(X;Y) \;\geq\; \Big(\sum_{(X,Y)\in\mathcal{M}} w_{XY}\Big) \log k - \mathcal{L}_{\text{total}}(\theta).$$

The right-hand side increases as $\mathcal{L}_{\text{total}}$ decreases, so minimizing the training objective maximizes a variational lower bound on the weighted sum of pairwise mutual informations, which establishes the theorem. In the closed-domain case ($\mathcal{P} = \emptyset$), the sum runs over the first three pairs and the bound specializes to the verifier Q.

Per-query interpretation (Remark in Section III-D). The InfoNCE-optimal critic satisfies $f^*(x, y) = \log \frac{p(x,y)}{p(x)\,p(y)} + c(x)$ for some function c depending only on the first argument [35], [43]. A trained critic evaluated on a single instance therefore estimates the pointwise mutual information up to an additive term independent of the second argument, and $Q' = \sigma\big(\sum_{(X,Y)} w_{XY} f_\theta(X, Y)\big)$ is a monotone aggregation of such pointwise estimates. This justifies using Q' as a per-query sufficiency score: it does not bound the (population) mutual information, which is the role of the training objective above. □

B. Cell-Link Constraint: Statement and Proof

The following proposition formalizes the search-space reduction and conditional coverage of cell-link conditioned retrieval, summarized in prose in Section IV-C.

Proposition 2:*[Cell-link constraint] Let $\bar{d}$ be the average cell-link out-degree. (a)* ***Search-space reduction.*** *For any table T, $|\mathcal{N}(T)| \leq \bar{d} \cdot |T.C|$, so conditioning on a retrieved table reduces the passage candidate pool from $M = |\mathcal{P}_{pool}|$ to $|\mathcal{N}(T)|$, typically by orders of magnitude. (b)* ***Conditional sufficiency.*** *If the gold passage is hyperlinked from the gold table, $p^* \in \mathcal{N}(T^*)$, and the retrieved table is correct, $\hat{T} = T^*$, then $p^* \in \mathcal{C}_{link}(\hat{T})$.* □

Proof: By definition, $\mathcal{N}(T) = \{p \in \mathcal{P}_{\text{pool}} : (T, p) \in \mathcal{E}\}$. Each cell in T contributes at most its hyperlink out-degree many passages to $\mathcal{N}(T)$. Since T has $|T|_{\text{cell}}$ cells and the average out-degree is $\bar{d}$, we have $|\mathcal{N}(T)| \leq \bar{d} \cdot |T|_{\text{cell}}$ (with equality when all hyperlinks point to distinct passages).

The reduction ratio is:

$$\frac{|\mathcal{P}_{\text{pool}}|}{|\mathcal{N}(T)|} \;\geq\; \frac{M}{\bar{d} \cdot |T|_{\text{cell}}}.$$

In the sparse-graph regime ($\bar{d} \cdot |T|_{\text{cell}} \ll M$), this ratio is $O(M/(\bar{d} \cdot |T|_{\text{cell}}))$.

Part (b): if $\hat{T} = T^*$ and $p^* \in \mathcal{N}(T^*)$, then $p^* \in \mathcal{N}(\hat{T}) \cap \mathcal{P}_{\text{pool}} = \mathcal{C}_{\text{link}}(\hat{T})$ by the definition of the candidate pool. □

C. Modality-Gap Calibration: Statement and Proof

The following proposition formalizes how the gate governs table–passage representation alignment, described qualitatively in Section IV-B.

Proposition 3:*[Modality gap calibration] Let $\boldsymbol{\mu}_T^{(l)}$ and $\boldsymbol{\mu}_P^{(l)}$ be the centroid embeddings of table and passage nodes at layer l. Under mean-aggregation message passing with mixing coefficient $\alpha \in (0, 1]$, the centroid distance $\|\boldsymbol{\mu}_T^{(l)} - \boldsymbol{\mu}_P^{(l)}\|$ is non-increasing in l, with contraction rate governed by the effective coefficient $\sigma(g)$: near initialization the gap is essentially preserved, and the contraction strengthens as the gate opens.* □

Proof: Consider the bipartite graph $\mathcal{G}$ with adjacency structure encoded by the normalized adjacency operator $\hat{A}$. Under mean-aggregation message passing with mixing coefficient $\alpha \in (0, 1]$, the update for table-type nodes is:

$$\mathbf{h}_T^{(l+1)} = (1 - \alpha)\mathbf{h}_T^{(l)} + \alpha \cdot \hat{A}_{TP}\, \mathbf{h}_P^{(l)},$$

and symmetrically for passage-type nodes. Writing the joint state as $\mathbf{H}^{(l)} = [\mathbf{h}_T^{(l)}; \mathbf{h}_P^{(l)}]$, the update is $\mathbf{H}^{(l+1)} = ((1-\alpha)I + \alpha\hat{A})\,\mathbf{H}^{(l)}$, where $\hat{A}$ is the symmetrically normalized bipartite adjacency.

The inter-type centroid distance is:

$$\delta^{(l)} = \|\boldsymbol{\mu}_T^{(l)} - \boldsymbol{\mu}_P^{(l)}\|.$$

Since $\hat{A}$ has spectral radius $\rho(\hat{A}) \leq 1$ for a normalized adjacency, and the mixing operator $M = (1 - \alpha)I + \alpha\hat{A}$ has spectral radius $\rho(M) = \max(|1 - \alpha + \alpha\lambda_i|)$ where λ_i are eigenvalues of $\hat{A}$, the second-largest eigenvalue of M satisfies $|1 - \alpha + \alpha\lambda_2| < 1$ for $0 < \alpha \leq 1$ and $|\lambda_2| < 1$ (which holds for connected bipartite graphs where $\lambda_2 < 1$).

The centroid difference lies in the subspace corresponding to eigenvalues $\neq 1$, so $\delta^{(l+1)} \leq |1-\alpha+\alpha\lambda_2| \cdot \delta^{(l)} < \delta^{(l)}$.

In SPARQ+, the gate mechanism (Eq. (13)) modulates the effective α: $\alpha_{\text{eff}} = \sigma(g)$, starting at ≈ 0.007 and increasing during training. At initialization, $|1-\alpha_{\text{eff}}+\alpha_{\text{eff}}\lambda_2| \approx 1$, so the gap is essentially preserved (cold-start safety). As the gate opens, the contraction becomes non-trivial.

□

D. Detailed Operators and corresponding prompts in SPARQ

In this section, we list a set of five operators, which are commonly used in SQL and LLM-based tableQA solutions. In the following, we present the specific operator and corresponding prompts. For simplicity, the demonstration examples are omitted from the presented prompts.

Base QA Operator o^{Base}: A baseline that provides LLM with the full table T and three few-shot demonstrations. Following CoT [73], it uses chain-of-thought, prompting LLM to divide TQA into several sub-tasks and lead to the final result. The prompt list is listed in Figure 16.

Column Extraction Operator o^{col}: SPARQ employs a column extraction operator, denoted as o^{col}, to identify query-relevant columns via two parallel paths: (1) direct enumeration via LLM reasoning $t^{\text{col}}(\cdot)$, and (2) SQL-based projection $S^{\text{col}}(\cdot)$. The union $o^{\text{col}}(q,T) = t^{\text{col}}(\cdot) \cup S^{\text{col}}(\cdot)$ forms the sub-table T^{col}, mitigating hallucination and syntax errors arising from single-path. Prompt is listed in Figure 11.

Row extraction Operator o^{row} SPARQ applies a row-based operator, O^{row}, to filter query-relevant rows via two complementary paths: (1) direct enumeration via LLM reasoning $t^{\text{row}}(\cdot)$, and (2) SQL-based selection $S^{\text{row}}(\cdot)$. The merged result $T^{\text{row}} = t^{\text{row}}(\cdot) \cup S^{\text{row}}(\cdot)$ forms the refined sub-table for subsequent reasoning. The prompt is listed in Figure 12.

Retrieval operator o^{Ret}**.** The retrieval operator O^{Ret} is a key innovation of SPARQ, introducing hybrid semantic retrieval that combines dense and sparse search. Given (q,T), the LLM expands q into multi-granular variants (general, balanced, specific), while rows and columns are linearized and scored using SBERT embeddings [33] as dense retrieval and BM25 [23] as sparse retrieval to measure pairwise similarity, and applies weighted reciprocal rank fusion [34] to merge them. The top-M rows and top-N columns are retained as T^{Ret}, with additional text or hyperlinks extracted for web tables. By bridging symbolic reasoning with RAG-style retrieval, SPARQ enables offline LLMs to efficiently access contextual evidence. The rewrite prompt is listed in Figure 13.

SQL execute operator. SPARQ employs an SQL execution operator, O^{SQL}, to perform symbolic computation by generating and executing SQL from (q,T). It supports filtering, comparison, and aggregation functions (e.g., COUNT, AVG), and enhances robustness by returning [UNKNOWN] when execution is infeasible, so that such operations are automatically skipped. Prompt is listed in Figure 15.

Training-Free Router $E_{4\text{B}}$ **and** $E_{30\text{B}}$**.** Following [82], we additionally test training-free router, which performance is evaluated in Section V-D. Prompt is listed in Figure 14.

E. Model Parameters of different models in SPARQ

The model parameters utilized within SPARQ are detailed in Table XIV. For non-GPU components, our implementation is based on `Python 3.10`, incorporating several key libraries for specific functionalities. We employ `sqlite3` and `sqlalchemy` for database construction and efficient SQL execution; `BM25` is used to handle sparse retrieval tasks; We utilize `pandas` for core data manipulation, enhanced by `pandarallel` to facilitate multithreaded data preprocessing.

F. Detailed Statistics of Safeguard Mechanisms

To assess the computational overhead of our safeguard mechanisms, we monitored the trigger frequency of both *Rollback* (internal re-routing) and *Fallback* (external SQL execution) across multiple benchmarks. As shown in Table XV, the Fallback mechanism is invoked sparingly, serving only as a necessary safety net for the most challenging queries.

G. Detailed optimization strategies

In TQA systems, the end-to-end inference pipeline typically comprises two distinct stages: (1) CPU-intensive tabular preprocessing and database operations; and (2) GPU-accelerated LLMs inference. A naïve implementation executes these two stages sequentially in a single thread, processing one query at a time adapted by most online model systems. While simple, this design leads to severe GPU underutilization due to frequent idle periods, commonly referred to as CPU-GPU bubbles, during which the GPU stalls while waiting for the CPU to complete preprocessing for the next query.

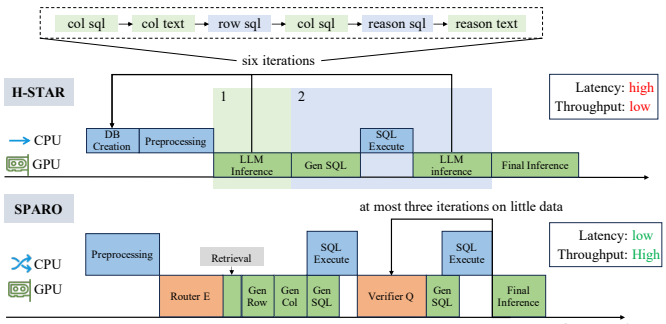


Fig. 6: Scheduling Pipeline of baseline H-STAR and SPARQ.

To address this inefficiency, we introduce a batched pipeline execution strategy that decouples and parallelizes the CPU-bound and GPU-bound stages of the workflow. Our design consists of two core components as depicted in Fig. 6.

Multi-threaded preprocessing. We parallelize tabular preprocessing using multithread and optimize the database operations through an asynchronous scheduling pipeline that enables streaming management of the SQL execution. To enhance robustness, we introduce an optimized parsing algorithm that automatically detects and filters out malformed SQL queries, thereby preventing erroneous queries from blocking the ex-

TABLE XIV: SPARQ component names, attributes, corresponding example design parameters, and model names.

SPARQ **Components**	**Attributes**	**Attributes**	**Model Name**
Embedding Model	Model size of the embedding model for o^{Ret}.	568M	BGE-M3
Backbone of Router	Model size of backboned router model E	568M	BGE-M3
Vector Dimensionality	The number of dimensions for each embedded vector	1,024	
Backbone of Check Model	Model size of backboned Check model V	560M	BGE-Reranker-Large
QA Model for $\mathsf{SPARQ}_{\mathsf{4B}}$	Model size of the LLM used for QA and operator generation.	4B	Qwen3-4B [83]
QA Model for $\mathsf{SPARQ}_{\mathsf{30B}}$	Model size of the LLM used for QA and operator generation.	30B	Qwen3-30B [84]

TABLE XV: Statistics of Rollback and Fallback mechanisms across different benchmarks. The *Rate* columns indicate the percentage of the total test set where the respective mechanism was triggered. Low Fallback rates confirm that the computationally expensive safeguard is invoked only sparingly.

Dataset	**Total Size**	**Rollback**		**Fallback**	
		Count	**Rate**	**Count**	**Rate**
WikiTQ	4,344	1,957	45.05%	528	12.15%
TableBench	886	173	19.53%	71	8.01%
TabFact	2,024	390	19.27%	295	14.57%
FetaQA	2,003	289	14.43%	60	3.00%
NIAT	3,989	920	23.06%	191	4.79%

ecution pipeline. This approach maximizes CPU throughput and reduces per-query preprocessing latency, especially in database-augmented TQA pipelines where SQL execution and robustness to malformed queries are critical.

Offline batching for GPU inference. Rather than dispatching each preprocessed query immediately, we batch structurally and semantically similar queries to maximize KV cache reuse and throughput. A resource-aware dynamic loader manages multiple GPU models, temporarily loading lightweight components (e.g., routing and verification) and releasing them upon completion to free memory for LLM inference. This offline batching exploits LLM parallelism, substantially improving GPU utilization and efficiency.

Batching combines multiple preprocessed queries into a single GPU operation, enabling concurrent processing that reduces CPU–GPU bubbles. For the offline TQA, this approach is far more effective than multithread (validated in Section V-D). Notably, our operator set is mutually independent which can execute in parallel, with results combined via intersection. However, the current scheduler does not leverage this property, which we leave for future work.

TABLE XVI: hyper-parameters for different operators in both $\mathsf{SPARQ}_{\mathsf{4B}}$ and $\mathsf{SPARQ}_{\mathsf{30B}}$. temp. represents temperature hyper-parameter for LLM inference.

Operators	**temp.**	**top_p**	**output_tokens**	**samples**	**examples**
o^{col}	0.7	0.8	2048	2	3
o^{row}	0.7	0.8	2048	2	3
o^{ret}	0.7	0.8	1024	1	3
o^{sql}	0.7	0.8	2048	3	4
o^{Base}	0.0	1.0	512	1	4
Final QA	0.0	1.0	512	1	4
$E^{\mathsf{4B}}, E^{\mathsf{30B}}$	0.0	1.0	128	1	3

H. Inference Parameter for SPARQ

The hyperparameters used to query LLM for each operator are listed in Table XVI. All datasets and methods within SPARQ adhere to this identical setting. It is worth noting that we additionally set `top_k=20`, `min_p=0` and `presence_penalty=1` to mitigate the issue of hallucination. For clarification, the term `samples` indicates the number of generations performed for the same query, while `examples` denotes the number of demonstrations for each query. Furthermore, E^{4B} and E^{30B} represent the training-free router instances discussed in detail in Section V-D.

TABLE XVII: Number of generated samples for different methods on dataset WikiTQ.

Method	**# samples / step**	**Total # samples**
DATER	Decompose Table: 40 Generate Cloze: 20 Generate SQL: 20 Query: 20	100
Chain-of-Table	Dynamic Plan ≤ 5 Generate Args ≤ 19 Query: 1	≤ 25
TabSQLify	Table Decompose: 1 Query: 1	2
H-STAR	Column Extraction: 4 Row Extraction: 4 Query: 2	10
ReACTable(s-vote)	Generate SQL: ≤ 10 Generate Python: ≤ 5 Query: 1	10.20
SPARQ (Ours)	Column Selection: 2 Row Selection: 2 SQL Execute: 3 Rewrite+Retrieval: 1 Query: 1	5.01

I. Analysis: Generated Samples

In Table XVII, we analyze the sample generation efficiency of various LLM-based solutions by comparing the average number of samples generated per query. This analysis serves as an extended discussion of the motivation presented in Section II and Figure 3. DATER utilizes self-consistency refinement at each step, resulting in 100 samples per query. In contrast, Chain-of-Table uses a more resource-efficient methods, denoted as `Dynamic Plan`,`Generate Args Plan` and `Query`, which significantly save the LLM generated sample number per query. H-STAR applies a fixed pipeline with intertwined steps of symbolic and textual extraction of column, rows and SQL generation. ReAcTable employs the ReACT method to generate multiple candidates with SQL and Python code in a sandbox environment, then conducts majority voting to integrate multiple results. TabSQLify generates the fewest samples, with one generation each for the table decomposition and query steps. While SPARQ use the dynamic route and check mechanism, to dispatch different level of queries into different subset of operators. Detailed discussion is listed in Section II.

J. Implementation Detail for baseline methods

We run H-STAR, ReAcTable, Chain-of-Table and TabSQLify using their official implementation and prompts in GitHub. In Table V, for baseline ReAcTable, Chain-of-Table and H-STAR, we conduct their native multithread implementation to 16 for best efficiency, while TabSQLify is in single thread, as it cannot natively support multithreading.

In Table V, we made minor modifications to ensure a fair comparison of the baselines. In detail, for H-STAR, Chain-of-Tables and TabSQlify, we use its default hyper-parameter designed for GPT-3.5-Turbo(*e.g.,* prompts, number of samples and demonstration examples), except that we set `top_p=0.7` and `top_k=0.8`, following the suggestion of qwen-3 series [83]. For ReAcTable, we use *s-vote* among its variant methods, which shows stability in their original paper.

In Table IX, the average prediction time and the I/O token throughput for all methods were calculated by client-side measurement of the server's response and I/O tokens across all requests. This was performed on an identical offline `vllm`-based server (OpenAI-capatible) using the same model `qwen3-4B`. To maximize generation speed, the `vllm` server was enabled with `FlashInfer` [85] and `fp8`-quantization [86] for both model weights and KV-Cache.

K. Detailed Time Breakdown Analysis

To provide a transparent and granular breakdown of latency sources, we recorded the execution time of each pipeline step. **Experimental Setup:** All time statistics reported in Table XVIII were collected on a **single NVIDIA RTX 4090 GPU**. To ensure a clear dissection of computational costs without the confounding factors of parallelism, the pipeline was executed in a **single-threaded sequential mode**. This strict setup allows us to precisely attribute the total runtime to specific modules and identify system bottlenecks.

The pipeline execution is categorized into five logical stages: (1) **Preprocessing** (Steps 1 & 3): Includes data loading and the construction of ephemeral SQLite databases for each table. (2) **Router** (Steps 2, 4 & 5): Encompasses the construction of routing prompts, the inference of the Router model, and the parsing of generated plans. (3) **Operator Execution** (Steps 6-9): Represents the core execution phase, divided into retrieval (*RAG*), context pruning (*Select Row/Col*), and symbolic reasoning (*SQL Generation & Execution*). (4) **Verification** (Steps 10 & 11): The safeguard mechanism that iteratively checks for missing information or execution errors. (5) **Final QA** (Steps 12 & 13): The final generation phase where the model synthesizes the answer based on the processed context.

Analyzing the breakdown reveals two critical insights regarding efficiency and task characteristics. First, the overhead of our proposed architectural components is minimal. The **Router** typically consumes less than 1.5% of the total time (e.g., 42.3s out of 3291.6s on NIAT), confirming that separating planning from reasoning incurs negligible latency cost. More importantly, the **Verification** overhead is exceptionally low, accounting for merely **0.6%** on WikiTQ and **0.2%** on TableBench. This empirically validates that our safeguard mechanism effectively amortizes its cost by triggering expensive fallbacks only when absolutely necessary. Second, the dominant latency sources align with dataset characteristics: WikiTQ, known for complex symbolic logic, spends the majority of time on *SQL Generation* (52.8%), whereas TabFact, which requires verifying long-context claims, heavily relies on *Select Operators* (48.9%) to prune tables. This demonstrates that SPARQ dynamically allocates computational resources to the operators most suited for the specific data modality.

L. Analysis of Iterative Code Generation Baselines

To benchmark SPARQ against pure code-generation approaches, we implemented two iterative baselines: **SPARQ-Py** and **SPARQ-SQL**. These methods prompt the LLM to generate Python or SQL code to solve the query. If execution fails (e.g., syntax errors or runtime exceptions), the model is reprompted with the error message up to 3 times. If all attempts fail, the system falls back to direct LLM generation.

Table XIX details their performance in terms of Accuracy, Latency (per query), and Execution Success Rate (ESR). **Key Observations:** (1) **Execution Instability:** The code generation methods suffer from inconsistent execution success rates. For instance, on NIAT (Qwen-4B), SPARQ-SQL only achieves a 28.85% execution success rate, heavily relying on the fallback mechanism. (2) **Performance Gap:** SPARQ consistently outperforms these baselines. On NIAT (Qwen-30B), SPARQ achieves **73.45%** accuracy with **0.95s** latency, surpassing SPARQ-Py (52.05%, 0.84s) in accuracy by a large margin and outperforming SPARQ-SQL (72.27%, 1.57s) in latency significantly. (3) **Efficiency Trade-off:** While SPARQ-Py is sometimes faster due to simple logic generation, it lacks the semantic depth required for complex reasoning, leading to lower accuracy. SPARQ strikes a superior balance by combining structural extraction with symbolic reasoning.

M. Complementary Roles of Operators

To further understand the contribution of different operator types, we conducted an ablation study removing specific operator groups, detailed in Table VIII. We observed that the removal of either table extraction operators or the Execute_SQL operator leads to a notable decline in performance, though these ablations impact different subsets of samples.

Specifically, removing **table extraction** severely impairs the LLM's ability to parse and reason over long tables, as the model loses the ability to simplify the context window. In contrast, removing the **Execute_SQL** operator deprives the LLM of a mechanism to verify its own calculations, resulting in frequent numerical reasoning errors. This confirms that SPARQ's hybrid design is essential for handling the diverse challenges of real-world TQA.

N. Error Analysis: Case Study

We provide additional case studies among different operator sequences in Figure 7 and Figure 8. Both cases are selected from the test split of the WikiTQ dataset.

TABLE XVIII: Time breakdown (in seconds) of different pipeline stages across benchmarks. **Preprocessing** includes data loading and database construction. **Router** includes query construction and inference. **Verification** sums the check model iteration and fallback SQL addition. **Final QA** includes prompt assembly and the final reasoning generation.

Dataset	Preproc.	Router	Operator Execution			Verify	Final QA	Total
	(Step 1,3)	(Step 2,4,5)	RAG	Select	SQL	(Step 10,11)	(Step 12,13)	Time
TableBench	11.6	12.3	0.0	148.0	279.4	2.4	598.9	1052.5
NIAT	5.0	42.3	9.9	679.6	1410.5	5.8	1138.4	3291.6
TabFact	27.8	21.8	9.5	1293.9	584.2	8.0	699.1	2644.1
WikiTQ	58.2	51.8	9.7	1007.8	2944.2	34.4	1469.0	5575.0

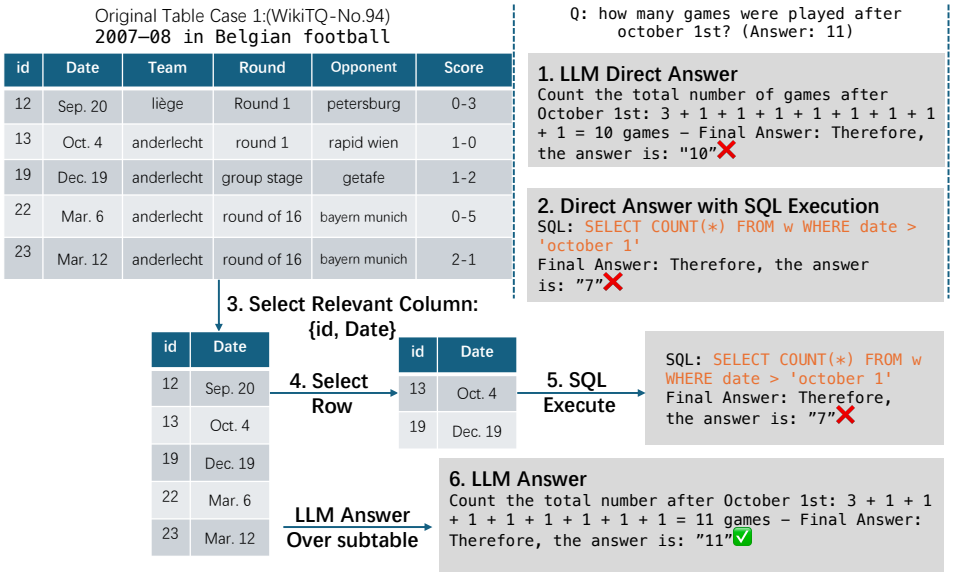


Fig. 7: Failure case in numerical and temporal TQA. This case asks, "how many games were played after October 1st?" (Correct Answer: 11). The LLM's direct answer yields "10" due to a counting error, exposing its weak native numerical ability. The direct SQL-based answer yields "7" because it fails to select games in next year(*e.g.,* row 23), highlighting the fragility of symbolic methods with non-normalized data.

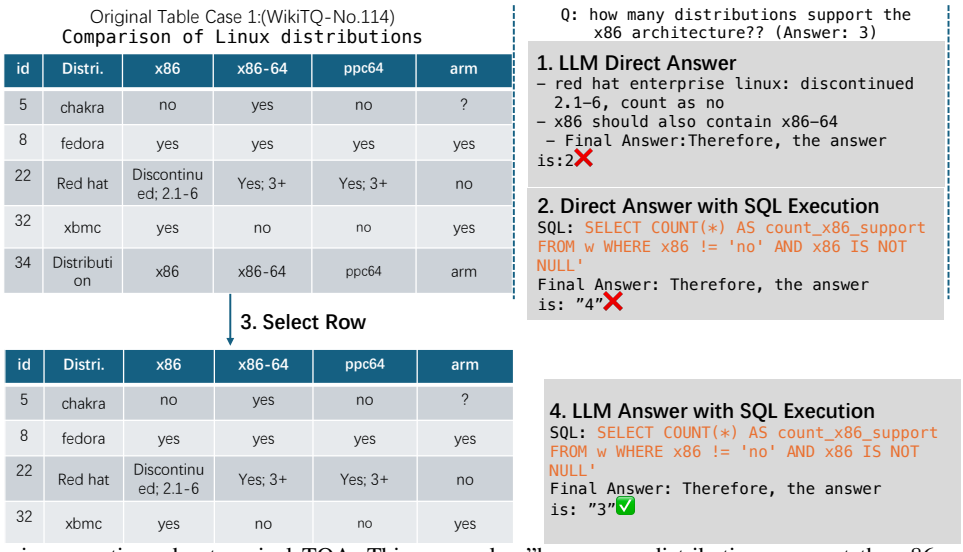


Fig. 8: Failure modes in semantic and categorical TQA. This case asks, "how many distributions support the x86 architecture?" (Correct Answer: 3). The LLM's direct answer incorrectly yields "2" by misinterpreting the context of "discontinued" and confusing "x86" with "x86-64", showcasing its shortage in semantic understanding with irrelevant context. Direct SQL-based answer incorrectly counts "4" due to its inability to filter out last row as header, again demonstrating the limited fault tolerance of symbolic methods for dirty tables.

O. Reasoning cliff test on TabelBench

We evaluated SPARQ on TableBench to show that long-form table deeply effects the TAQ performance. As shown in Table XX, the Rouge-L score exhibits a non-linear trend: performance peaks at moderate table lengths (400–550 tokens, Rouge-L = 0.5437) and gradually declines as tables become longer, returning to baseline levels ($\geq$1200 tokens). This "reasoning cliff" suggests that excessively long tables may

TABLE XIX: Performance of iterative code generation baselines. **Acc.** denotes Rouge-L for TableBench and Exact Match (EM) for NIAT. **Lat.** is the average per-query latency in seconds. **ESR** (Execution Success Rate) indicates the percentage of queries successfully solved by code execution without reverting to fallback.

Dataset	Method	Accuracy/EM	Latency	ESR
	Backbone: Qwen3-4B			
TableBench (N=886)	SPARQ	49.08	1.07s	-
	SPARQ-Py	41.65	0.88s	45.71%
	SPARQ-SQL	35.19	1.35s	60.95%
	Backbone: Qwen3-30B			
	SPARQ	52.00	1.32s	-
	Execute-Py	49.46	0.76s	74.83%
	Execute-SQL	45.75	1.13s	81.72%
	Backbone: Qwen3-4B			
NIAT (N=3,989)	SPARQ	66.58	0.72s	-
	Execute-Py	44.95	0.52s	53.75%
	Execute-SQL	64.08	0.56s	28.85%
	Backbone: Qwen3-30B			
	SPARQ	73.45	0.94s	-
	Execute-Py	52.05	0.84s	62.55%
	Execute-SQL	72.27	1.58s	48.12%

TABLE XX: Table size trends on TableBench with Qwen3-30B

Tokens	Rouge-L
$0-300$	0.4750
$400-550$	0.5437
$550-800$	0.5267
$800-1200$	0.5011
$1200+$	0.4753

overwhelm the model's contextual reasoning capacity.

P. Performance on FeTaQA

We evaluated SPARQ with training-based and training-free baselines on FeTaQA. The results detailed in Table XXI show SPARQ outperforms the baseline in the vast majority of cases.

Q. The Verification Module Minimizes Information Loss from Aggressive Routing

Fig. 9 illustrates the average number of table cells retrieved per query by SPARQ compared to baseline methods. We observe that a router model, due to its limited input and representational capacity, is naturally biased towards selecting more complex and aggressive extraction paths, which introduces a significant risk of information loss. The verification model, Q, counteracts this tendency by triggering a rollback mechanism for erroneous selections that result in an insufficient context. This corrective action was activated in 51.9% of table size

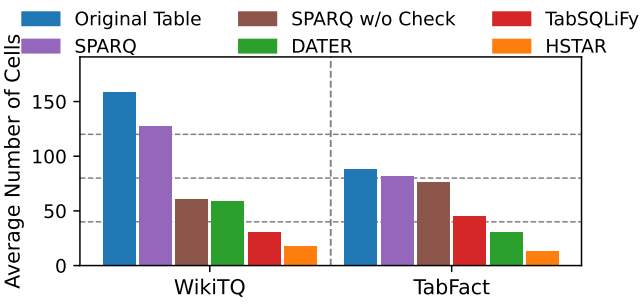


Fig. 9: Comparison of average table cells retrieved for final QA for SPARQ_{4B} and baselines with online LLM baselines.

TABLE XXI: Experimental results on FeTaQA.

Model	Rouge-1	Rouge-2	Rouge-L
Approaches requiring training			
T5-small	0.55	0.33	0.47
T5-base	0.61	0.39	0.51
T5-large	0.63	0.41	0.53
Approaches without training			
DATER	0.66	0.45	0.56
ReAcTable	**0.71**	0.46	0.61
TabSQLify	0.58	0.35	0.48
$\text{HSTAR}_{\text{gpt3.5}}$	0.62	0.39	0.52
Chain-of-Tables	0.66	0.44	0.56
Ours			
SPARQ_{4B}	0.60	0.36	0.49
SPARQ_{30B}	0.69	**0.51**	**0.65**

on the WikiTQ dataset and 7.4% on TabFact. Although this makes SPARQ's retrieval strategy appear more conservative than the baselines (i.e., it retains more cells on average), this is a direct consequence of prioritizing information sufficiency.

R. Performance with online LLMs

To verify the scalability of SPARQ beyond offline settings, we evaluated its performance using the state-of-the-art online model, **Qwen3-Max** [87], on the challenging TableBench dataset. The results, summarized in Table XXII, highlight three critical advantages:

- **Superior Accuracy:** SPARQ achieves a Rouge-L score of **0.55**, significantly outperforming the agent-based ReAcTable (0.40) and the decomposition-based H-STAR (0.35). This confirms that our routing mechanism effectively enhances even the most capable frontier models.
- **Drastic Cost Reduction:** A key bottleneck of agentic frameworks is token consumption. ReAcTable consumes **48.0M** tokens due to its verbose thought loops. In contrast, SPARQ consumes only **3.8M** tokens ($\approx 12\times$ reduction) while achieving higher accuracy, making it a far more economically viable solution for enterprise deployment.
- **Low Latency:** SPARQ maintains a latency of **2.67s**, which is comparable to direct inference and dramatically faster than ReAcTable (15.73s) and H-STAR (17.23s). This efficiency stems from our non-iterative routing design, avoiding the high latency penalty of multi-turn agent interactions.

S. Analysis of the robustness of SPARQ *to table width*

To evaluate the robustness of SPARQ to tables width, we additionally evaluate SPARQ on two synthesized variants: TableBench_num (avg. 6.63 columns, up to 20) and TableBench_cols (avg. 42.31 columns, up to 49). There variants are constructed via feature derivation, with results detailed in Table XXIV. SPARQ achieves consistent performance across both datasets on Qwen3-4B and Qwen3-30B (Rouge-L 0.6488 and 0.7269 vs. 0.6638 and 0.7579). This robustness is enabled by the Column-Filter operator, which prunes irrelevant attributes and prevents the LLM from being overwhelmed by excessive table width.

TABLE XXII: Experimental results on TableBench with Qwen3-Max.

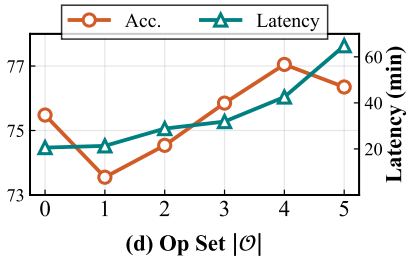
Method	Qwen3-4B		Qwen3-30B		Qwen3-Max		
	Rouge-L	Latency	Rouge-L	Latency	Rouge-L	Latency	Token Consumption
H-STAR	0.02	2.63	0.26	3.74	0.35	17.23	5.1M
ReAcTable	0.05	**1.57**	0.39	1.90	0.40	15.73	48.0M
SPARQ	**0.49**	1.71	**0.52**	**1.32**	**0.55**	**2.67**	**3.8M**

TABLE XXIII: Experimental results on TableBench with local deployment.

Method	Qwen3-4B		Qwen3-30B		Token Consumption
	Rouge-L	Latency	Rouge-L	Latency	
H-STAR	0.02	2.63	0.26	3.74	5.1M
ReAcTable	0.05	**1.57**	0.39	1.90	48.0M
SPARQ	**0.49**	1.71	**0.52**	**1.32**	**3.8M**

TABLE XXIV: Experimental results on the robustness of SPARQ to table width variations. The metrics of Qwen3-4B and Qwen3-30B are Rouge-L.

Dataset	Avg. Columns	Max Columns	Qwen3-4B	Qwen3-30B
TableBench_num	6.63	20	0.6488	0.7269
TableBench_cols	42.31	49	0.6638	0.7579



(d) Op Set $|\mathcal{O}|$

Fig. 10: Additional Hyperparameter Analysis of operator set expansion for SPARQ_{4B} on WikiTQ dataset.

T. Hyper-parameter: impact of Operator Set Expansion ($|\mathcal{O}|$).

Fig. 10(d) evaluates the cost-benefit trade-off of operator expansion. The system achieves peak accuracy (77.05%) with the inclusion of the o^{SQL} operator (Index 4), which offloads symbolic logic effectively. However, further expanding to o^{Python} (Index 5) causes a performance drop and a sharp latency spike. This indicates that unconstrained code execution is not always beneficial for offline models, given their limited code-generation success rate (69%) and lower tolerance to noisy data types compared to structured SQL.

This appendix provides supplementary experimental results and analyses omitted from the main paper due to space constraints.

U. Full Offline Comparison Table

V. Cross-domain Generalization and Operator-sequence Distribution

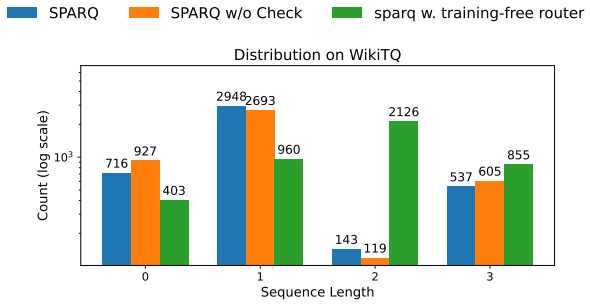


Fig. 17: Distribution of $|\mathcal{S}|$ on WikiTQ by SPARQ_{4B}.

W. Open-Domain Progressive Ablations and Diagnostics

Open-domain configuration. Table XXVII lists the full open-domain configuration. The GNN uses L=3 HGT layers with hidden dimension h=1024; the gate is initialized at $g=-5$ ($\sigma(g_0)\approx 0.007$) so retrieval starts from the dense embedding and opens the structural channel only as it proves useful. The GNN is trained with InfoNCE (Eq. (15)) at temperature τ_t=0.05 against hard negatives from the dense retriever's top-K. The cross-encoder reranker rescores the top-20 RRF candidates. The passage budget is K_{pas}=15, and the passage-term weights in Q' (Eq. (19)) are $\delta=\epsilon$=0.5.

Passage budget sweep. Table XXVIII sweeps K_{pas} on the passage-recoverable subset of OTT-QA (gold table fixed, 35B reader). Recall exhibits diminishing returns: K=5 already captures 81% of the K=80 gain, and K=15 closes most of the remainder. This confirms that the cell-link neighborhood supplies high-precision candidates, so a small budget within the structural prior suffices; enlarging post-retrieval budgets does not address the residual table-ranking bottleneck.

Neighborhood-size stratification. Stratifying OTT-QA by the gold table's cell-link neighborhood size reveals the boundary of the two-stage "constrain then rank" design. On small neighborhoods ($\leq$20 links, n=923), the structural prior narrows the candidate pool to a size where BM25 ranking is sufficient, and SPARQ+ reaches 0.728 EM. On large neighborhoods (>50 links, n=222), the prior still reduces the search space from

TABLE XXV: Full unified offline comparison on TabFact, WikiTQ, NIAT and TableBench datasets using the same offline backend LLM with different training-free baseline methods. All offline models are deployed with 2 RTX 4090 GPUs. Latency is the average seconds per query measured using 16 threads for all methods if supported.

Method	Qwen3-4B								Qwen3-30B							
	TabFact		WikiTQ		NIAT		TableBench		TabFact		WikiTQ		NIAT		TableBench	
	Acc	Lat.	Acc	Lat.	EM	Lat.	Rouge-L	Lat.	Acc	Lat.	Acc	Lat.	EM	Lat.	Rouge-L	Lat.
ReAcTable	25.84	0.40	0.58	0.59	0.95	0.93	0.05	1.57	66.90	0.56	33.47	0.63	26.08	**0.56**	0.39	1.90
TabSQLify	6.23	3.14	63.58	5.08	55.67	4.58	0.24	4.94	45.06	2.96	72.26	4.46	67.98	3.68	0.31	3.85
H-STAR	60.33	1.66	8.98	2.06	13.99	1.49	0.02	2.63	88.24	2.26	70.33	2.48	56.37	2.16	0.26	3.74
Chain-of-Tables	17.74	1.20	49.15	2.02	63.02	1.29	0.30	1.60	83.75	2.12	58.31	1.96	69.96	1.22	0.37	1.97
SPARQ (Ours)	**91.30**	**0.31**	**77.03**	**0.49**	**66.58**	**0.72**	**0.49**	**1.07**	**92.19**	**0.33**	**79.79**	**0.55**	**73.45**	0.95	**0.52**	**1.32**

TABLE XXVI: Cross-domain generalization. Diagonal entries (bold) indicate in-domain performance.

Test Dataset	Training Source		
	TableBench	WikiTQ	TabFact
TableBench(Rouge-L)	**0.491**	0.465	0.471
WikiTQ(Acc,%)	74.30	**77.03**	75.10
TabFact(Acc,%)	87.50	89.47	**91.30**

TABLE XXVII: Open-domain hyperparameters of SPARQ+.

Stage	Parameter	Value
Stage 1 (GNN)	HGT layers L	3
	hidden dim h	1024
	gate init g	-5
	InfoNCE temp. τ_t	0.05
Stage 1 (fusion)	reranker candidates	top-20 (RRF)
Stage 2	passage budget K_{pas}	15
Stage 3 (Q')	passage weights δ, ϵ	0.5, 0.5

millions to hundreds, providing high recall; however, ranking noise within the expanded pool accumulates, and EM drops to 0.599. HELIOS's final refinement phase reaches 0.662 in this regime, because its multi-round rescoring absorbs the within-pool noise that a single BM25 pass cannot resolve. This pattern is consistent with the budget sweep above: the structural constraint supplies high-precision candidates, while effective within-pool ranking remains a complementary requirement.

Output-length profile. Within the S1✓ bucket of Table XI, the CoT gain peaks at moderate generation lengths (+18.3 EM for 150–1,000 output tokens), while generations exceeding 1,000 tokens form a failure signature (EM 0.109, below Direct; half of them are S1× queries). An overlong reasoning chain thus signals insufficient evidence at zero extra cost, providing a runtime hook for the adaptive routing mechanism to trigger fallback or re-retrieval.

Stage-1 signal non-redundancy. The three Stage-1 retrieval signals remain non-redundant: pairwise Kendall's $\tau \leq 0.28$ across their rank lists, and 10.2% / 3.5% / 11.8% of gold tables are found at rank 1 exclusively by BM25 / dense / GNN, respectively. Signal fusion rather than budget expansion is therefore the path to closing the remaining table-miss gap.

Post-retrieval refinement has diminishing returns. HELIOS's phase ablation under an identical scorer shows its final refinement phase costs 17h50m on OTT-QA while decreasing EM by 0.18 pp (58.94→58.76), indicating that iterative post-retrieval processing does not address the table-ranking bottleneck. The productive directions are instead sharper Stage-1 ranking, whose gains the reasoning side amplifies (Table XI), and answer synthesis, which adds +5.6 EM with no additional retrieval passes.

TABLE XXVIII: Cell-link passage budget sweep on OTT-QA (passage-recoverable subset, gold table fixed, 35B reader).

K_{pas}	5	15	40	80
EM	0.515	0.639	0.687	0.700

X. *Hyper-parameter Analyses*

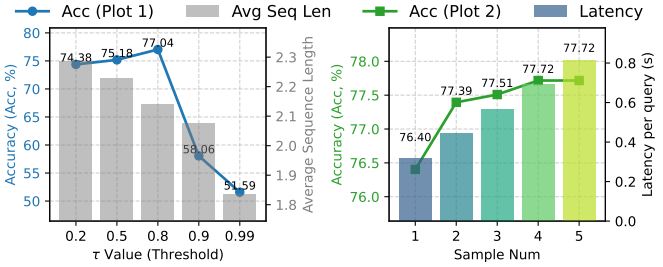


Fig. 18: Hyperparameter analysis for SPARQ_{4B}.

Threshold τ for the verification model. As shown in Fig. 18 (left), τ trades off accuracy and operational complexity. A permissive threshold (*e.g.,* τ=0.2) approves most operator sets and increases the average number of operators, but slightly degrades accuracy due to error propagation. An overly tight threshold (*e.g.,* τ=0.99) triggers frequent rollbacks and substantially hurts accuracy.

Sampling numbers. Fig. 18 (right) shows that increasing the sample number for $o^{\text{row}}, o^{\text{col}}, o^{\text{SQL}}$ can correct formatting errors and modestly improve accuracy, but incurs a significant latency overhead, especially for SQL generation.

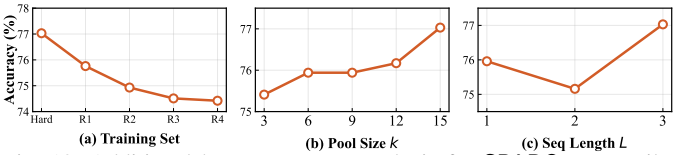


Fig. 19: Additional hyperparameter analysis for SPARQ_{4B} on WikiTQ.

Robustness to training distribution. We compare training on hard samples against random sampling of the same size. While hard samples yield the best accuracy, the router remains robust when trained on random subsets.

Sensitivity to search configuration (k and L). Reducing the operator pool size degrades accuracy, while blindly increasing the candidate count can lead to overly aggressive routing; a moderate candidate count offers a better balance.

Instruction for o^{col}

(system message) You are given the table schema, and the table along with the corresponding statement. Write a simple SQLite program for selecting the required columns only, to help answer the question correctly. The SQLite program need not directly answer the question. Assume you always have enough information when executing the SQLite. Fuzzy match data if unsure.
(task description)
1. Plan:
- Identify critical values and ranges from the table related to the statement.
- Make use of your domain knowledge to find the correct approach to solve the question.
- Always select the column with special aggregate values like 'total'.
2. Retrieval:
- Generate a simple SQL program extracting the relevant columns
- SQL: SELECT COLUMNS FROM w;
- Evidence: f_{col}(column names)
(output format)
Response Format: Begin your response with 'Output' and include:
- Plan: Write the plan for column extraction along with a reasoning chain
- Retrieval: Write a simple SQL query
Evidence: f_{col}(column names)
(instruction) Before you return the answer, review your outputs and make sure you have followed all the given instructions.
(input)

```
CREATE TABLE figure skating at the asian winter games(
    row_id int,
    rank int,
    nation text,
    gold int,
    silver int,
    bronze int,
    total int)
/*
SELECT * FROM w;
row_id rank nation gold silver bronze total
0 1 china 13 9 13 35
1 2 japan 7 10 7 24
2  3 uzbekistan 1 2 3 6
3 6 total   24 23 26 73
*/
columns: ['row_id', 'nation', 'gold', 'silver', 'bronze', 'total']
statement: the number of gold and silver medals are equal
```

Fig. 11: Column Selection Prompt o^{col}

Instruction for o^{row}

(system message) You are given the table schema, and the table along with the corresponding statement. Your task is to write an SQLite program to create a subtable to help answer the question correctly. The SQLite program need not answer the question. Try to use fuzzy-match for values if you are not sure about the values.
(task description)
Process:
1. Decompose: Find all the subtasks from the main question.
2. Retrieval: Generate an SQLite program for each subtask and then combine using an 'OR' statement.
(output format)
Response Format: Your response must strictly have the following components:
- Decompose: Divide the question into smaller manageable sub-tasks; one task for each condition
- Retrieval: A basic SQLite program for each condition followed by a combined SQLite
NOTE
- Strictly use the 'OR' statement to when you combine the multiple conditions in the final SQLite program.
Add rows if not present in the response.
Rows: f_row([rows])
(demonstration)

```
<input>
CREATE TABLE 2005 pba draft(
  row\_id int,
  pick int,
  player text,
  country\_of\_origin text)
/*
3 example rows:
SELECT * FROM w LIMIT 3;
row\_id pick player country\_of\_origin
1 2 alex cabagnot united states
2 3 dennis miranda philippines
*/
statement: the player after alex cabagnot is from philippines
<output>
Take a deep breath and think step by step
1. Decompose:
  - Part 1: Identify the row for alex cabagnot
   Response 1: SELECT * FROM w WHERE player = 'alex cabagnot';
  - Part 2: Identify the player after alex cabagnot
   Response 2: SELECT * FROM w WHERE pick = (SELECT pick FROM w WHERE player LIKE
   '%alex cabagnot%') + 1;
  - Part 3: Check if the player after alex cabagnot is from the Philippines
   Response 3: SELECT * FROM w WHERE country\_of\_origin = 'philippines';
2. Retrieval:
   # Using 'OR' to combine the conditions from #1, #2, #3
   SQL: SELECT * FROM w WHERE player = 'alex cabagnot' OR pick = (SELECT pick FROM
   w WHERE player LIKE 'alex cabagnot') + 1 OR country\_of\_origin = 'philippines';
```

(input)

```
CREATE TABLE jeev milkha singh(
 row_id int,
 tournament text,
 events int,
 cuts made int)
/*
SELECT * FROM w LIMIT 3;
row_id tournament  events cuts made
0 masters tournament 3 2
1 us open 4 3
2 the open championship 2 1
*/
columns: ['row_id', 'tournament', 'events', 'cuts made']
statement: the number of cut made exceeds the number of event by 2
```

Fig. 12: Row Selection Prompt o^{row}

Instruction for o^{Ret}

(ROLE) You are an expert in data retrieval strategies for hybrid search systems. Your goal is to rewrite a user's question into a set of three distinct queries, each optimized for a different aspect of a hybrid (dense + sparse) retrieval pipeline.
(CONTEXT)
The user provides an '[Original Query]' and a '[Table Sample]'. The table sample is a small representative extract from a much larger table and will always include a 'row_id' or similar index column. Your task is to use the content and schema of the sample to generate the queries.
(TASK: GENERATE THREE QUERY VARIANTS)
Based on the user's query and the table sample, generate three rewritten queries with increasing specificity:
1. [**GENERAL**] Query (Optimized for Dense/Vector Search):** A descriptive, semantic sentence capturing the user's core intent.
2. [**BALANCED**] Query (Optimized for Hybrid Search):** A concise query blending semantic intent with the most critical column names.
3. [**SPECIFIC**] Query (Optimized for Sparse/Keyword Search like BM25):** A "keyword-stuffed" query that aggressively lists relevant column names and specific cell values from the sample. This should be a space-separated list of terms, not a grammatical sentence.
(Output Format)
You MUST provide the output in the following exact format:
[GENERAL]: Your general, semantic query here
[BALANCED]: Your balanced, semantic + keyword query here
[SPECIFIC]: Your specific, keyword-stuffed query here
(demonstration)

```
<input>
/*
col : rank | cyclist | team
row 0 : alejandro valverde (esp) | caisse d'epargne
row 1 : alexandr kolobnev (rus) | team csc saxo bank
row 2 : davide rebellin (ita) | gerolsteiner
row 3 : paolo bettini (ita) | quick step
row 4 : franco pellizotti (ita) | liquigas
row 5 : denis menchov (rus) | rabobank
row 6 : samuel sánchez (esp) | euskaltel-euskadi
row 7 : stéphane goubert (fra) | ag2r-la mondiale
row 8 : haimar zubeldia (esp) | euskaltel-euskadi
row 9 : david moncoutié (fra) | cofidis
*/
columns: ['rank', 'cyclist', 'team']
Q: which country had the most cyclists finish within the top 10?
</input>
<output>
[GENERAL]: Determine which country is represented by the highest number of cyclists in the top
10 ranking by counting the nationalities mentioned in the cyclist column.
[BALANCED]: Count cyclists by country from the 'cyclist' column to find the most frequent
nationality.
[SPECIFIC]: country count cyclist rank esp rus ita fra alejandro valverde (esp) alexandr
kolobnev (rus) davide rebellin (ita) samuel sánchez (esp) haimar zubeldia (esp)
```

(input)

```
table caption: 2007 New Orleans Saints season
/*
col : game site | result/score
row 0 : rca dome | l 41 \ 10
row 1 : raymond james stadium | l 31 \ 14
row 2 : louisiana superdome | l 31 \ 14
row 4 : louisiana superdome | l 16 \ 13
row 9 : louisiana superdome | l 37 \0 29
*/
columns: ['game site', 'result/score']
Q: what number of games were lost at home?
```

Fig. 13: Rewrite Prompt for o^{Ret}

Instruction for training-free router E_{4B}

(system message) Given a query and a table, determine the necessary operations to answer the query. The order of the operators in the output list represents their importance, with the first operator being the most crucial.
(task description)
The possible operations are:
'Base': No specific operation is needed; the answer can be found by directly observing the table.
'Execute_SQL': Required for complex calculations or aggregations (e.g., COUNT, SUM, AVG, GROUP BY).
'Select_Column': Needed when the query specifically asks for information from a subset of columns.
'Select_Row': Needed when the query requires filtering the table to find specific rows based on certain conditions.
'RAG': Employ a Retrieval-Augmented Generation (RAG) system. This involves rewriting the query and performing a semantic search to find relevant content. This is useful when the query's intent isn't directly translatable to a simple SQL condition and may require external knowledge.
(Output Format) Your output must be a single JSON list. Here is an example template for the output:
["Select_Column", "Select_Row", "Execute_SQL"]
(demonstration)

```
<input>
Q: how many german racers finished the race?
Table Content:
CREATE TABLE 00__German_motorcycle_Grand_Prix (
pos text,
no int,
rider text,
manufacturer text,
laps int,
time_retired text,
grid int,
points int
)
/*
All rows of the table:
SELECT * FROM 00__German_motorcycle_Grand_Prix;
pos  no           rider manufacturer  laps time_retired  grid  points
1   75   mattia pasini       aprilia  27.0   39:44.091   3.0    25.0
2   19 álvaro bautista       aprilia  27.0       +0.01   2.0    20.0
3   52     lukáš pešek         derbi  27.0      +0.111   1.0    16.0
... (33 rows omitted) ...
*/
columns: ['pos', 'no.', 'rider', 'manufacturer', 'laps', 'time/retired', 'grid', 'points']
<output>
["Base", "Select_Row"]
```

(input)

```
Q: which country is represented by the most drivers?
Table Content:
CREATE TABLE _00__Gran_Premio_Telmex (
pos int,
no int,
driver text,
team text,
laps int,
time_retired text,
grid int,
points int
)
/*
All rows of the table:
SELECT * FROM _00__Gran_Premio_Telmex;
pos no              driver                  team  laps time_retired  grid  points
1  1 sébastien bourdais    newman/haas racing    66  1:51:31.146     2      34
2  9     justin wilson               rusport    66     +pt3.528s     1      29
3  5          will power        team australia    66    +pt46.536s     4      26
... (8 rows omitted) ...
*/
columns: ['pos', 'no', 'driver', 'team', 'laps', 'time/retired', 'grid', 'points']
```

Fig. 14: Training-Free Router Prompt E_{4B}

Instruction for o^{SQL}

(system message) Your task is to determine whether you need an SQLite program to solve the question. You must understand the question and identify if it involves any calculations. The final SQLite program must be simple.
(task description)
Final Output: If the question has any mathematical component, then you must write an SQLite program to extract a subtable to answer the question. Otherwise, simply return 'None'
(Output Format) Begin your response with 'Output: ' and always include the following:
- Final Output: A step-by-step reasoning followed by 'None' if SQL is not required otherwise the SQLite program as the solution.
Exclude aggregate values like 'total', 'average', etc from your SQLite programs.
Follow all instructions before returning the answer. Be careful. Think step by step.
(demonstration)

```
<input>
CREATE TABLE asian games (
   row_id int,
   rank int,
   lane int,
   player text)
/*
All rows of the table:
SELECT * FROM w;
row_id   rank  lane  player
0     5  olga tereshkova (kaz)
1     6  manjeet kaur (ind)
2     3  asami tanno (jpn)
*/
columns: ['row_id','rank','lane','player']
Q: which country has the highest number of athletes?
Final Output:
We need to count the occurrences of athletes from each country. Since this involves
mathematical operations, we will use SQLite to solve this.
 - Step 1: Extract the country from the player column, assuming it is enclosed within parentheses.
   SELECT SUBSTR(`player`, INSTR(`player`, '(') + 1, INSTR(`player`, ')') - INSTR(`player`,
   '(') - 1) AS country FROM w;
 - Step 2: Count the occurrences of athletes from each country and find the country with the
 highest count.
   SELECT SUBSTR(`player`, INSTR(`player`, '(') + 1, INSTR(`player`, ')') - INSTR(`player`,
   '(') - 1) AS country, COUNT(*) AS num_athletes FROM w GROUP BY country
   ORDER BY num_athletes DESC LIMIT 1;
SQL: SELECT SUBSTR(`player`, INSTR(`player`, '(') + 1, INSTR(`player`, ')') - INSTR(`player`,
'(') - 1) AS country, COUNT(*) AS num_athletes FROM w GROUP BY country ORDER BY num_athletes
DESC LIMIT 1;
```

(input)

```
<input>
CREATE TABLE Fabrice Santoro(
row_id int,
name text,
career\nwin-loss text)
/*
All example rows:
SELECT * FROM w;
row_id   name  career\nwin-loss
0 australian open   22{18
1 indian wells 17{20
2 total 39-38
*/
columns: ['row_id', 'name', 'career\nwin-loss']
statement: how many games did he win in total?
```

Fig. 15: SQL Execute Operator Prompt o^{SQL}

Instruction for o^{Base}

(system message) You are an expert on table data. You must use the table data and the additional evidence to answer the given question.
(task description)
Procedure:
- Divide the main statement into sub-tasks and answer each sub-task
- Based on the answers, check whether the statement is supported by the table
NOTE
Please be extremely careful, pause, make sure all instructions have been followed and only then output the answer.
(Output Format)
Begin your response with 'Output: ' and always include the following:
- Decompose: Divide the main question into sub-tasks and answer each sub-task
- Final Answer: Strictly output as a short phrase starting by 'therefore, the answer is: "AnswerName1", "AnswerName2"...' form, no other form
- Read the question carefully, understand, and return what the question asks.
- Be careful, make sure you have followed all instructions and only then return the output.
(demonstration)

```
<input>
table caption: 2007 New Orleans Saints season
/*
col : game site | result/score
row 0 : rca dome | l 41 / 10
row 1 : raymond james stadium | l 31 / 14
row 2 : louisiana superdome | l 31 / 14
row 4 : louisiana superdome | l 16 / 13
row 9 : louisiana superdome | l 37 / 29
row 10 : reliant stadium | l 23 / 10
row 12 : louisiana superdome | l 27 / 23
row 15 : louisiana superdome | l 38 / 23
row 16 : soldier field | l 33 / 25
*/
columns: ['game site', 'result/score']
Q: what number of games were lost at home?
<output>

Here is an additional evidence to help the answering proces
Additional Evidence:
/*
col : games_lost_at_home
row 0 : 5
*/
Using the table and the additional evidence to answer the question
1. Decompose:
    - #1: From the additional evidence, number of games lost at home = 5
          From the table, counting the occurrences of "louisiana superdome" in the
          "game site" and 'result/score' for loss column = 5
2. Final Answer: Therefore, the answer is: "5"
```

(input)

```
<input>
table caption: Matthew Morrison
/*
col : year | title
row 0 : 2007 | music and lyrics
row 1 : 2007 | dan in real life
row 2 : 2007 | i think i love my wife
*/
columns: ['year', 'title']
Q: what movies other than 'music and lyrics' was morrison involved with in 2007?
```

Fig. 16: Base QA Operator Prompt o^{Base}